

Software-as-a-Graph: When Does Graph Learning Improve Cascade-Impact Ranking in Publish–Subscribe Systems?

Ibrahim Onuralp Yigit^{a,*}, Feza Buzluca^a

^a*Department of Computer Engineering, Istanbul Technical University, Istanbul, 34469, Turkey*

Abstract

Publish–subscribe middleware obscures the paths through which failures propagate, yet architects must identify critical components before deployment, when no telemetry exists. We present Software-as-a-Graph (SaG), which derives explicit dependency graphs from deployment manifests to investigate when graph learning improves cascade-impact ranking. We compare graph neural networks, hybrids, and analytical and learned simulator approximations with training-free rankings on twelve synthetic architectures under leave-one-scenario-out evaluation against three failure simulators, and zero-shot on five hand-authored models of open-source systems. A reference criterion distinguishes rankings that restate a simulator’s propagation rule from those that predict it.

On a reachability simulator, graph attention on the derived dependency graph reaches Spearman’s $\rho = 0.748$, +0.072 above the same model with reverse edges on the raw multigraph, whereas on that graph the registered primary contrast against a training-free baseline was null. The learned model is matched by counting direct dependents ($\rho = 0.764$), suggesting little predictive signal beyond explicit dependencies. For a queue-flow simulator requiring 12.7 CPU-hours to label the corpus, a rate-weighted first-order approximation reaches $\rho = 0.830$ in milliseconds, matching or exceeding a learned approximation trained on simulator labels ($\rho = 0.799$; exploratory). Hybrids that correct a closed-form prior significantly outperform the training-free baseline but not their base learners, and learned models transfer zero-shot better than that baseline ($\rho \approx 0.81$ vs. 0.53), although dependency counts still rank higher.

All findings are conditional on simulation fidelity. In this simulated corpus, representation mattered more than model complexity, highlighting the importance of deriving meaningful dependency representations and evaluating learned methods against analytical alternatives aligned with the simulator.

Keywords: Dependency graphs, cascading failures, publish–subscribe, graph neural networks, graph learning, software architecture, dependability

1. Introduction

1.1. Motivation and Problem

Large distributed systems now often use asynchronous publish-subscribe (pub-sub) middleware. Examples include ROS 2 for autonomous driving [1], Apache Kafka for enterprise event streams [2], DDS in cyber-physical systems [3], MQTT in IoT [4], and cloud-native microservices [5, 6]. Pub-sub separates producers and consumers by space, time, and synchronization [7]. Instead of direct references, components communicate via topics and brokers, and Quality-of-Service (QoS) policies set at deployment control reliability, durability, priority, and deadlines.

This decoupling also makes it harder to see how failures spread. Since publishers and subscribers are not directly connected, problems like outages, blocking, and backpressure can move through undetected paths

*Corresponding author

Email addresses: yigiti@itu.edu.tr (Ibrahim Onuralp Yigit), buzluca@itu.edu.tr (Feza Buzluca)

involving brokers, shared topics, colocated hosts, and shared libraries [8, 9]. Failures usually appear in two ways. In sequential cascades, a slow subscriber fills up a broker queue and gradually starves its publishers [10]. In simultaneous blasts, a crash in a shared library or a host outage can bring down all colocated services at once. These issues are not visible in architecture diagrams or static call graphs. The best time to handle these risks is before deployment, during design and continuous integration [11, 12], when runtime telemetry is unavailable. Therefore, architects must identify the most critical application services, topics, and links using only configuration manifests.

Current practices leave a serious architectural gap between service-level static analysis and distributed system dependability. A distributed system might have no compile-time bugs in any service but still face major outages due to hidden single points of failure, unbuffered queue bottlenecks, or mismatched QoS contracts [13, 14]. Approaches like the Architecture Tradeoff Analysis Method (ATAM) depend on manual input [15]. Static code analysis inspects services in isolation [16, 17]. Chaos engineering [18] needs a working cluster, and pre-production tests like fault injection [19] and dependency tracing [20] require runnable environments. In continuous integration, heavy fault injection or chaos experiments use significant computing power and energy, so pre-deployment static analysis is a greener option if it is accurate enough [21, 22, 23]. Graph-based representations have long supported reliability and defect analysis [24, 25], and graph learning has recently been applied to network criticality [26, 27] and to microservice architectures [28]. Such models can combine structural and contractual information, but it remains unclear whether their predictive performance arises from the learning algorithm or from the dependency representation it operates on, and when learning adds value beyond analytical methods derived from the same dependencies. Software-as-a-Graph (SaG) addresses this question by deriving explicit dependency graphs from deployment manifests and enabling a controlled comparison of analytical, hybrid and learned approaches to cascade-impact ranking before deployment.

1.2. The Software-as-a-Graph (SaG) Approach

Software-as-a-Graph (SaG) is a pre-deployment analysis framework for event-driven architectures (Figure 1). It models an architecture as a typed, directed multigraph over Applications, Brokers, Topics, Execution Hosts and Shared Libraries (Section 3.1), and derives an explicit `DEPENDS_ON` dependency graph through publish-subscribe rules (Section 3.3). On this basis, it supports analytical, hybrid and learned approaches to cascade-impact ranking (Sections 4 and 5.2). This study uses it to determine when learned models add value beyond rankings derived directly from explicit dependencies.

Rankers never read simulator output: their node features are computed on the analysis graph, and learners pass messages over either the raw multigraph or the derived dependency graph. Labels come from three simulators that run on the raw structural topology (Section 4.3) and encode different failure notions: structural reachability (I^*), discrete-event queue saturation (I_{dyn}) and multi-criteria fragmentation (I_{comp}). To avoid depending on one simulator’s assumptions, every ranker is evaluated against all three. Because simulators and rankers read the same architecture, some analytical rankings restate a simulator’s own propagation rule. A reference criterion (Section 4.4) reports these as references that measure circularity rather than as predictors. To bound, though not remove, this circularity, we also report partial correlations after removing the primary simulator, and we time the cost of running each simulator directly (Section 6.4). All conclusions are strictly conditional on simulation fidelity; validation against real failures in physical distributed environments remains future work (Section 7.5).

Findings in brief. Explicit dependency representations provide most of the predictive signal for the reachability and queue-flow simulators.

Making dependencies explicit enables graph learning. On the raw multigraph, every relation points away from Applications, so message passing never reaches the scored nodes, and the registered primary contrast (a heterogeneous graph transformer against the training-free baseline on that graph) was null ($\Delta\rho = +0.069$, $p = 0.266$). On the dependency graph, attention-based learners improve: graph attention reaches Spearman $\rho = 0.748$ against the reachability simulator (GAT-P-QoS), compared with 0.635 on the raw multigraph (GAT-QoS), whereas the typed transformer does not improve in the tested configuration. Edge direction does not account for the gain: passing every raw-graph edge in both directions recovers +0.041 (not significant), and the derived graph adds +0.072 beyond it (Holm $p = 0.014$).

The same representation also enables simple analytical rankings that the learned models do not exceed. Counting direct dependents, which is publish–subscribe afferent coupling, reaches 0.764 and is not statistically distinguishable from the best learned model; ablations show that the learner approaches this count by drawing on its degree features (-0.136 without them). On the expensive queue-flow simulator (12.7 CPU-hours to label the corpus), a rate-weighted first-order approximation reaches 0.830 in milliseconds and exceeds a gradient-boosted approximation trained on the simulator’s labels (0.799; exploratory). Declared publication rates and payload sizes carry measurable signal only on this simulator; declared QoS policies and QoS-weighted scores add none.

Learning adds measurable value only relative to a weak comparator. Hybrids that correct a closed-form prior significantly outperform the training-free baseline ($+0.103$ and $+0.130$, each on 11 of 12 folds), also after its articulation defect is corrected ($+0.136$ and $+0.107$), but they do not differ from their own base learners, and given the direct-dependent count as their prior they reproduce that count. Learned models transfer zero-shot to five stylized models of open-source systems better than the training-free baseline ($\rho \approx 0.81$ vs. 0.53), mainly by separating inert from active components. Dependency counts rank higher still, and ordering among active components remains weak ($\rho_{>0} \leq 0.342$). All three simulators are well approximated by low-order functions of the dependency graph, so the benchmark did not exercise a regime in which learning could exceed an analytical ranking aligned with the simulator.

In practice, the results favor training-free dependency analysis for routine use. Recovering 80% of the true critical set requires reviewing about 40–45% of Applications, so rankings suit prioritized review rather than blocking gates (Section 7.4).

1.3. Research Questions

Establishing when learning is unnecessary is as informative as showing when it succeeds. Software engineering research has repeatedly shown that simple metrics can match the performance of more complex models when the right information is available [25, 29, 30, 31]. Analytical rankings can also be easier to explain, validate, and run in Continuous Integration and Continuous Deployment (CI/CD) pipelines. We therefore evaluate learned models against analytical rankings derived from the same dependencies, not only against structural baselines. Four research questions guide the study:

- **RQ1 (Ranking accuracy):** *How accurately do analytical, hybrid, and learned approaches rank components by cascading-failure impact on unseen architectures, and how close do they come to reference rankings that restate a simulator’s propagation rule?*
- **RQ2 (Sources of predictive performance):** *At matched model capacity, which factors contribute most to ranking performance: the dependency representation, degree information, relation typing, Quality-of-Service (QoS) information, or the model family (attention or sum aggregation; graph neural network or gradient-boosted trees)?*
- **RQ3 (Transfer):** *How well do learned models trained on synthetic architectures transfer zero-shot to stylized models of five open-source systems, hand-authored by a single modeler, and does their ordering of components that actually propagate failures hold?*
- **RQ4 (Cost):** *What do analytical and learned approaches cost at CI/CD time in latency and estimated energy, which stage dominates, and how does this compare with running each simulator directly?*

The evaluation uses a version-controlled analysis plan. Only the plan’s two co-primary contrasts are confirmatory, and both were null. We registered the matched 2×2 , the hybrids, the reference counts, the dependency-graph learners, the full-population queue-flow labels, and the sensitivity arms as planned extensions. We registered each before its own tests ran but after we knew the main result, and we report them as secondary results. The rest of the analysis is exploratory (Section 5.3). One change from the plan, not applying the registered nested hyperparameter selection, is disclosed and tested. The replication repository records the revision history and shows how these four research questions fit into the plan.

1.4. Contributions

This paper makes five contributions:

1. **Dependency derivation for publish–subscribe architectures** (Section 3). A method for deriving explicit dependency graphs from deployment manifests using typed architectural rules, extending the Application-level rule of [32] with library-mediated and infrastructure dependencies.
2. **A reference criterion for simulator-based evaluation** (Section 4.4). A criterion that distinguishes rankings that restate a simulator’s propagation rule from rankings that predict its outcomes.
3. **A controlled comparison of analytical, hybrid, and learned ranking methods** (Sections 5.2 and 6.1–6.3). A study that separates the effects of dependency representation, degree information, relation typing, QoS information, and model family on cascade-impact ranking.
4. **Analytical and learned approximations of an expensive simulator** (Sections 5.2 and 6.1). A rate-weighted first-order approximation of a queue-flow simulator (Eq. (7)), compared with learned approximations trained on the simulator’s labels.
5. **A reproducible benchmark, cost analysis and practitioner guidance** (Sections 5.1, 6.4 and 7.4). A reproducible benchmark of seventeen architectures with three simulators, a like-for-like cost analysis, and recommendations for pre-deployment cascade-impact analysis.

A previous conference paper [32] introduced the publish–subscribe multigraph, the Application-level subscriber-to-publisher dependency (Rule 1 of Table 2), and a closed-form betweenness–articulation score similar to the training-free baseline (Eq. (5), unweighted and with weights 0.7/0.3), which it validated against a reachability-loss simulation on synthetic topologies and two ROS 2 benchmarks. This paper adds the Library and infrastructure rules, QoS weighting and the QoS edge encoding, the reference criterion, the learned and hybrid rankers, the analytical and learned simulator approximations, LOSO and zero-shot evaluation, the second and third oracles, the matched control, and the cost analysis. The closed-form score family validated in [32] is the weakest analytical ranker in the present evaluation, which adds reference rankings, the Application population and three oracles; the present results supersede that paper’s implicit recommendation of it.

Section 2 reviews related work. Sections 3 and 4 present the model and the rankers. Sections 5 and 6 cover the evaluation. Section 7 discusses the findings and threats, and Section 8 concludes.

2. Related Work

2.1. Dependability and Failure-Impact Analysis of Distributed Systems

Runtime dependability methods like broker clustering, backpressure, autoscaling, failover, and chaos engineering [18] require a running cluster, can interrupt service, and consume significant computing resources. This resource use is a major concern in green software engineering and Green AI [21, 22, 23], where the carbon footprint of large execution clusters in continuous integration encourages lightweight, energy-efficient static alternatives. Pre-production methods, such as service-level fault injection testing [19] and microservice dependency localization with PageRank (MicroRank [20]), offer lighter options before deployment. Benchmark platforms with cataloged, replayable faults, like Train-Ticket [33], help measure the impact of failures on a running deployment, which this study has not yet validated (Section 7.6). Architecture-based reliability prediction has a long history. Yacoub and Ammar’s method for architecture-level reliability risk analysis [24] was one of the first to combine component dependency graphs with complexity and failure severity to rank components by risk. State, path, and additive reliability models reviewed by Goseva-Popstojanova and Trivedi [34] are classic examples, as is the Palladio Component Model [35].

Error propagation has also been analyzed at the architecture level. Abdelmoez et al. [36] estimated error propagation probabilities between architectural components, and Popic et al. [37] added them to architecture-based reliability models of component-based systems. Cortellessa and Grassi [38] analyzed how error propagation affects system reliability, and Hiller et al. [39] profiled how data errors propagate across software modules (EPIC). In safety-critical systems, model-based failure analysis is standard practice, including Hierarchically Performed Hazard Origin and Propagation Studies (HiP-HOPS [40]) and the Architecture Analysis & Design Language (AADL) Error Model Annex [41]. These frameworks derive propagation paths from structural descriptions, serving as conceptual precursors to SaG’s dependency projection. However,

these approaches need component failure-mode annotations, propagation probabilities or operational profiles, which deployment manifests do not carry; SaG asks a narrower question using only the manifests.

Telemetry-based methods find faults in running microservices. Examples include Seer [42], MicroRCA [43], and recent GNN-based root-cause frameworks like Eadro [44], which Zhang et al. [45] thoroughly review. In synchronous microservices, failures can spread because of thread-pool exhaustion, RPC timeouts, and retry storms [33]. In publish-subscribe systems, issues spread through queue saturation and message starvation. All these methods need data from a live system, but SaG works before any code is run. SaG’s focus is similar to change impact analysis [46] and dependency management in microservice fleets [47]. Ranking by impact is related to learning-to-rank defect prediction [48], which explicitly optimizes the ranking measure, as our listwise loss does (Section 4.2).

Rankings of failure impact are often evaluated against a failure model or simulator rather than against observed outages. Such evaluation creates an interpretive challenge: a ranking may agree with a simulator because it predicts the simulator’s behavior, or because it encodes the simulator’s propagation rule. We are not aware of prior work that separates these two cases explicitly.

2.2. Static Analysis and Architectural Dependency Metrics

Static code analysis (SCA) tools like SonarQube [16] measure complexity [49], cohesion, and coupling [17] within individual services to identify modules likely to have defects [50, 51]. Software engineering defect prediction has long compared complex graph measures with simple structural metrics. For example, Zimmermann and Nagappan [25] studied network analysis on dependency graphs, while Premraj and Herzig’s replication study [29] showed that simple code and coupling metrics frequently perform as well as network measures. SCA cannot detect inter-service messaging, broker saturation, or issues that spread across hosts. Architecture recovery tools rebuild system-level structure from code [52], and HAROS [53] checks ROS systems before launch. SaG’s static system analysis (SSA) uses the declared topology instead, spreading code-level metrics across architectural dependencies. Architecture-level coupling metrics count a service’s consumers, such as afferent coupling [54], the Absolute Importance of the Service [55], and service fan-in [56]. In publish-subscribe systems, these counts are not simply a single edge lookup, since consumers reach producers only via topics, brokers, and shared libraries. They require a typed two-hop query (publisher to topic to subscriber), which SaG’s dependency rules define clearly. Because this count corresponds to the first propagation wave of the reachability oracle used here, this paper reports it as a reference that restates the oracle, not as a predictor (Section 4.4). This helps teams find structural anti-patterns [57, 58] and architectural technical debt [59] in CI/CD [60]. These studies suggest that much of the information about architectural criticality can be recovered directly from explicit dependency structure. They leave open whether learned methods add predictive information beyond such dependency-based analyses.

2.3. Multi-Criteria Weighting

Combining QoS policies or failure effects into a single, auditable weight is a multi-criteria decision problem. The Analytic Hierarchy Process (AHP) [61] is the standard method, and SaG uses it for the topic weights in Section 3.2 and the composite oracle in Section 4.3. AHP’s consistency ratio can find inconsistent judgments, but it does not detect matrices that have been filled in to match a chosen answer. This study notes that distinction for its weights.

2.4. Graph Learning on Dependency Structures

Closed-form indices [62, 63, 64, 65], including one used as this study’s training-free baseline, and cascade models of network robustness [10, 8, 9] assume homogeneous, usually undirected graphs. This means a single untyped score can mix up different elements like topics, libraries, and hosts. Learned node-importance methods, such as FINDER [26], DrBC [27], and PowerGraph [66], also assume homogeneity, as does learned approximation of betweenness and closeness [67]. Homogeneous GNNs like Graph Attention Networks (GAT [68]) ignore relation identity unless it is added as a feature. Softmax attention averages over neighbors and cannot count them, while sum aggregation can [69, 70]; theoretically, standard message-passing GNNs are strictly limited in counting substructures without explicit features [71]. Since the main reference here is

neighbor count, this difference matters (Section 6.2). Heterogeneous GNNs, like the Heterogeneous Graph Transformer (HGT [72]), learn transformations specific to each relation. However, extensive benchmarking by Lv et al. [73] demonstrated that heterogeneous GNNs often fail to outperform well-tuned homogeneous baselines like GAT, directly paralleling our typing ablation findings. More broadly, Shchur et al. [74] showed that GNN rankings are sensitive to data splits and tuning, while Huang et al. [75] showed that label propagation combined with shallow models can outperform GNNs on transductive node classification. We test both types with the same capacity (Section 6.2). Khodabandeh et al. [28] use graph attention on microservice call graphs to predict upcoming interactions. In contrast, we predict the effect of removing a node from a declared topology. GNN explainers like GNNExplainer [76] describe models using their internal features. SaG’s explanation layer instead names ISO/IEC 25010 quality sub-characteristics [77] and a remediation class for each flagged component, but this is not evaluated here. These studies show that graph learning can exploit structural information in software and network graphs. Their evaluations typically compare learned models with selected baselines; fewer examine whether performance originates from the graph representation or from the learning algorithm applied to it.

2.5. Positioning of This Study

This study builds on findings that simple baselines can perform as well as more complex learning methods [30, 31], and that network measures on dependency graphs [25] add little over simple coupling metrics once those are included [29]. In line with Green Software Engineering and Green AI principles [21, 22, 23], the cost of learned models in continuous integration should also be weighed against that of simple static analyses.

The study differs from prior work in three ways. First, it derives explicit dependency graphs from publish–subscribe deployment manifests rather than starting from an existing architectural graph. Second, it evaluates analytical, hybrid, and learned ranking methods using the same representation and asks where their ranking performance originates. Third, it introduces a reference criterion that distinguishes rankings that restate a simulator’s rule from those that predict its outcomes. Rather than asking whether graph learning can rank cascade impact, the study asks when it adds measurable value beyond rankings derived directly from the explicit dependency structure. More broadly, the study contributes to understanding the limitations of AI techniques for reliability analysis by identifying when apparent predictive accuracy reflects restatement of the evaluation simulator rather than learned skill.

3. The Software-as-a-Graph (SaG) Architectural Model

Figure 1 shows the full SaG framework architecture. The shared front end handles Architecture-as-Code manifests, builds a typed multigraph, derives a logical dependency layer that makes runtime failure paths explicit, and extracts typed node properties. This representation supports graph-learning pipelines and makes software dependencies explicit, so analytical, hybrid, and learned cascade-impact rankings can be computed from the same architectural representation (Sections 4 and 5.2). This common representation enables controlled comparisons between methods that differ in complexity but operate on identical dependency information. The proposed explanation layer reads the same properties separately, with no shared parameters.

3.1. Multigraph Definition

A distributed system is described as a typed, weighted, directed multigraph:

$$\mathcal{G} = (V, E, \tau_V, \tau_E, w_V, w_E) \quad (1)$$

where:

- $V = V_{\text{app}} \cup V_{\text{broker}} \cup V_{\text{topic}} \cup V_{\text{host}} \cup V_{\text{lib}}$ holds the five entity types $\mathcal{T}_V$ of Table 1; V_{host} denotes physical or virtual *Execution Hosts*.
- E is the set of directed edges, and $\tau_V : V \rightarrow \mathcal{T}_V$ and $\tau_E : E \rightarrow \mathcal{T}_E$ assign entity and relation types.

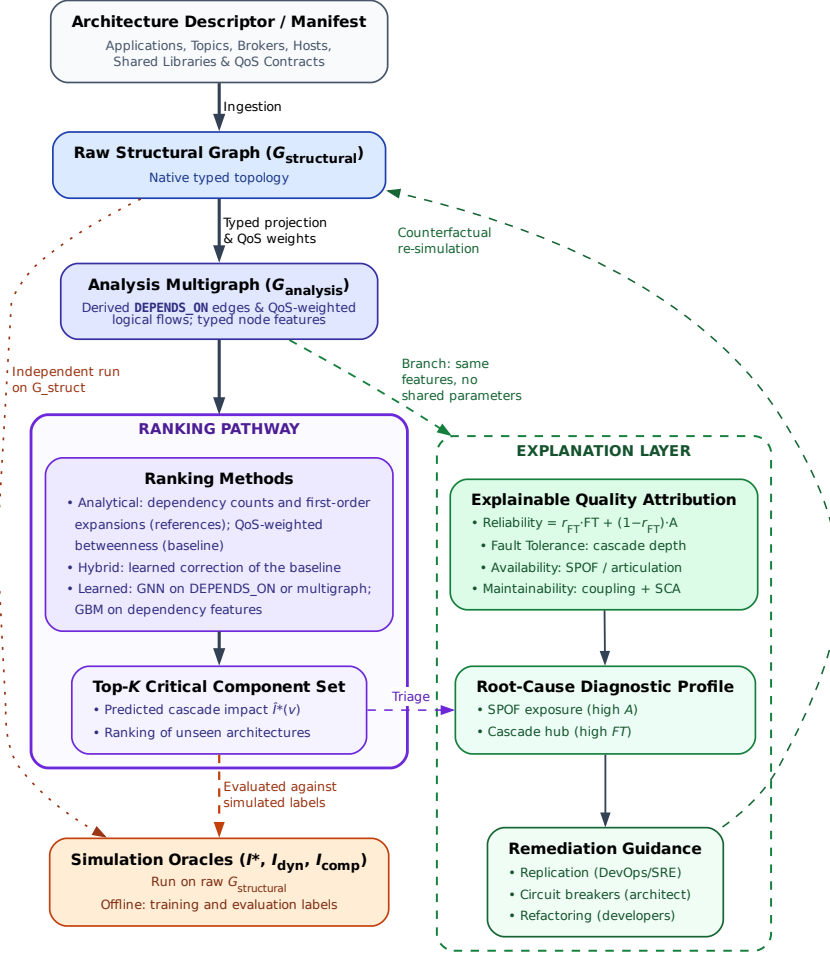


Figure 1: End-to-end architecture of the SaG framework. The ranking pathway runs down the center: manifest ingestion, typed structural multigraph ($G_{\text{structural}}$), DEPENDS_ON logical dependency projection (G_{analysis}) with typed node properties, the ranking methods (analytical, hybrid, and learned; Figure 3), and the ranked critical set. The same dependency graph supports all three ranking methods. The dashed edge marks the simulation oracles, which operate strictly on $G_{\text{structural}}$, provide training and evaluation labels offline, and do not participate in inference. The proposed explanation layer (not evaluated) reads G_{analysis} , shares no parameters with the rankers, and applies to components after they have been ranked; no ranker output flows into it.

- $w_V : V \rightarrow (0, 1]$ and $w_E : E \rightarrow (0, 1]$ weight entity criticality along with connection strength. In the implementation evaluated here, w_V of an Application is a power mean of the QoS weights $w(t)$ (Section 3.2) of the topics it publishes or subscribes to, and a Library takes the largest weight among its own topics and its consuming Applications, amplified by its fan-out. A code-quality penalty (CQP), computed from lines of code, cyclomatic complexity, instability and cohesion, does not enter w_V ; it and its four inputs appear only as node features of the learned rankers (Section 3.5).

The raw multigraph is denoted $G_{\text{structural}}$. The simulation oracles run on it, and the raw-graph learners pass messages over its edges. The analysis graph G_{analysis} is $G_{\text{structural}}$ augmented with the derived DEPENDS_ON edges (Section 3.3) and the typed node properties computed from them (Section 3.5). Its Application–Library DEPENDS_ON projection is the graph the analytical rankings and the -P learners read. Supplementary Section S1 lists the notation.

Scope of Evaluation and Entity Roles. While the multigraph formalizes five entity types, this study concentrates its empirical study specifically on *Applications* (V_{app}). In this work, the term *Application* is adopted to align with publish–subscribe standards (such as OMG DDS and ROS 2 participant nodes); conceptually, it corresponds directly to individual microservice containers, autonomous service agents, or

communicating daemon processes in distributed architectures. Applications represent the developer-authored, code-level services that are often changed, refactored, and tested in CI/CD pipelines, so they are the main focus for pre-deployment triage. Brokers, Topics, Execution Hosts, and Libraries are also modeled and included because they are important for understanding indirect failure propagation. For example, we cannot analyze message routing, shared fault-domain colocation, or library-related failures without them. However, ranking infrastructure components like Brokers and Hosts is more relevant to operational provisioning than to software triage at commit time, so we treat this as a separate operational issue. Consequently, the empirical evaluation in this paper focuses strictly on ranking developer-authored Applications (V_{app}); evaluating broker queue bottlenecks and physical host outages remains an important operational extension (Section 7.6).

Table 1: Entity types and structural edge types in the SaG model.

Entity Type ($\mathcal{T}_V$)	Architectural Role	Concrete System Examples
Application (V_{app})	Process producing/consuming messages	ROS 2 node, Kafka microservice, MQTT client
Broker (V_{broker})	Message routing and queuing intermediary	RabbitMQ exchange, Mosquitto, EMQX broker
Topic (V_{topic})	Named logical communication channel	<code>/sensor/lidar</code> , <code>orders.payment.completed</code>
Execution Host (V_{host})	Physical or virtualized execution environment	Bare-metal server, Kubernetes worker, Cloud VM
Library (V_{lib})	Shared software package or runtime dependency	Kafka client, OpenCV, Protobuf runtime
Structural Edge ($\mathcal{T}_E$)	Direction	Semantic Meaning
PUBLISHES_TO / SUBSCRIBES_TO	App/Library $\rightarrow$ Topic	Component publishes to / consumes from topic
ROUTES	Broker $\rightarrow$ Topic	Broker manages and routes topic traffic
RUNS_ON	App/Broker $\rightarrow$ Host	Process is hosted on host
CONNECTS_TO	Host $\rightarrow$ Host	Network link between hosts
USES	App $\rightarrow$ Library	Application links to shared library

3.2. Quality-of-Service Link Weighting

A link’s strength is conceptually determined by its Quality-of-Service (QoS) contract. For example, a RELIABLE topic with TRANSIENT_LOCAL durability creates a stronger connection between services than a BEST_EFFORT telemetry stream. Each topic t has an overall weight $w(t) \in (0, 1]$ that combines its QoS policies (reliability, durability, priority) with payload size and publication frequency. We set the reliability, durability, and priority weights employing an Analytic Hierarchy Process (AHP) pairwise-comparison matrix, which we defined independently and did not adjust to fit a target vector ($CR = 0.016$, non-degenerate). The framework’s other AHP matrices, including the one behind the I_{comp} weights (Section 4.3), were instead filled in to reproduce weight vectors chosen in advance, so their consistency ratios carry no evidential weight.

Although we formalize this multi-criteria weighting to capture declared middleware contracts, the evaluation in Section 6 tests whether this contractual information improves cascade ranking. It does not: on the reachability and queue-flow simulators, declared QoS policy parameters carried no measurable signal. For the reachability simulator this null is largely fixed by design, because I^* barely reads QoS (Section 4.3); the queue-flow simulator does read QoS policies, so there the null may be informative, although its sensitivity to them was not measured (Section 4.3). On the reachability oracle, more than half of the apparent QoS effect came from the QoS-weighted in-degree feature, and the remainder was not significant (Section 6.2). On the queue-flow oracle, the gains came from declared publication rates and payload sizes, while QoS-policy shares added -0.005 ($p = 0.68$; Section 6.1). We retain the full AHP formalization for architectural completeness and reproducibility; in this corpus, omitting QoS policy weighting did not reduce ranking accuracy. The dependency counts reported in this paper are unweighted and do not use $w(t)$.

3.3. Logical Dependency Projection (DEPENDS_ON)

Structural edges do not directly show how failures spread. For example, a subscriber depends on a publisher, but pub-sub topologies have no direct edge between them. SaG therefore derives an explicit semantic relation, DEPENDS_ON, from the dependent to the dependency: if the target fails, the source is affected. The derived edges expose the architectural pathways through which failures may propagate, and the resulting dependency graph is the common input for analytical rankings, hybrid methods, and graph-learning models. Table 2 lists the rules.

Table 2: The DEPENDS_ON logical dependency projection rules. [†]Defined for completeness; not exercised by the Application-level evaluation.

Rule	Dependency Category	Structural Pattern (Dependent $\rightarrow$ Dependency)	Derived Weight (w)
1	app_to_app	Subscriber $\rightarrow$ Publisher (via shared Topic)	$1 - \prod_{t \in T} (1 - w(t))$
2 [†]	app_to_broker	Publisher/Subscriber $\rightarrow$ Broker routing its topics	$1 - \prod_{t \in T} (1 - w(t))$
3 [†]	host_to_host	Host $\rightarrow$ Host (lifted from inter-host app dependencies)	$\max_{u \in \text{hosted}(h_1), v \in \text{hosted}(h_2)} w_E(u \rightarrow v)$
4 [†]	host_to_broker	Host $\rightarrow$ Broker (lifted from hosted app dependencies)	$\max_{u \in \text{hosted}(h)} w_E(u \rightarrow b)$
5	app_to_lib	Application $\rightarrow$ Shared Library it USES	$H(w_V(\text{app}), w_V(\text{lib}))$
6 [†]	broker_to_broker	Broker $\leftrightarrow$ Broker (shared fault-domain colocation, symmetric)	$w_V(\text{host})$

Rules 1 and 2 combine topics T that connect a pair using probabilistic union [78], so that having multiple failure paths increases the connection strength. Rule 5 uses the harmonic mean $H(x, y) = 2xy/(x + y)$ [79]. Rules 3 and 4 assign dependencies to hosts by taking the maximum value.

Sequential cascades and simultaneous blasts. Rule 1 describes sequential cascades, where if a publisher fails, its subscribers are affected because they do not receive messages. Rule 5 describes simultaneous blasts, where if a library crashes, all applications that depend on it fail at the same time. Neither mechanism is stated explicitly in a deployment manifest; each rule makes one of them visible. Shared-library dependencies are included because they create simultaneous blast failures that message-flow topology alone does not represent. Libraries that publish or subscribe are also endpoints for Rule 1, and an Application can reach a library’s topics through Rule 5. InDeg, Reach, Topo-QoS and the graphs the -P learners read apply Rule 1 to direct subscriptions only (Eq. (2)). The node-property extraction, which produces the in-degree feature every learner receives, additionally follows up to three USES links, so an Application that subscribes through a library it uses counts as a Rule 1 dependent in that feature only. Rules 2, 3, 4, and 6 cover infrastructure and broker dependencies (see Table 2), which define physical host failure domains and broker routing bottlenecks. Since this evaluation focuses on Application-level code, these four rules are not used in the ranking benchmarks but are included for completeness.

Remark 1 (the dependency count is a typed two-hop count). Let G_{flow} be the Application–Library projection (Rules 1 and 5) and v an Application. By construction,

$$\text{InDeg}(v) = |\{u \neq v : \exists t \in V_{\text{topic}}, (v, t) \in \text{PUBLISHES_TO} \wedge (u, t) \in \text{SUBSCRIBES_TO}\}|. \quad (2)$$

Since Rule 5 edges end at Libraries, every edge into an Application is a Rule 1 edge $u \rightarrow v$, which exists exactly when $u \neq v$ (an Application or a Library) directly subscribes to a topic that v publishes; in the presence of multiple parallel topics between the same pair, parallel topics collapse into a single directed dependency edge in G_{flow} .

The right side of Eq. (2) represents publish–subscribe afferent coupling (fan-in, or the Absolute Importance of the Service [54, 55]), and is a typed two-hop query on the raw multigraph. InDeg can be computed directly without projection; the projection defines which typed query to use. We checked this equality on all seventeen corpus graphs, with a maximum absolute difference of 0. We measure the value of the derivation separately: including Rule 5 raises the agreement of transitive reach with I^* by +0.058 (9 of 12 folds, Holm $p = 0.0068$; Section 6.1). Since the primary oracle’s first propagation wave matches this set (Section 4.4), InDeg and its transitive version Reach are used as reference points in this paper, not as predictors.

3.4. Dual Graph Views

The **structural graph** $G_{\text{structural}}$ is the raw deployment topology. The **analysis graph** G_{analysis} adds the DEPENDS_ON edges and code metrics (see Figure 2). Every ranker’s node features are computed on G_{analysis} ; the learners pass messages either over the edges of $G_{\text{structural}}$ or over the DEPENDS_ON projection (Table 4), and no ranker reads simulator output (Section 4.4). The two views also differ for a learner: on $G_{\text{structural}}$, every relation points away from Applications, so forward message passing does not reach them. In the DEPENDS_ON graph, each Application receives messages from its dependents (Section 6.2). The raw multigraph is therefore also a comparison point in the evaluation: it exposes communication structure but not cascading dependencies. Because the two views also differ in edge direction, Section 6.2 adds a raw-graph learner that passes every message in both directions, so that dependency semantics can be separated from direction.

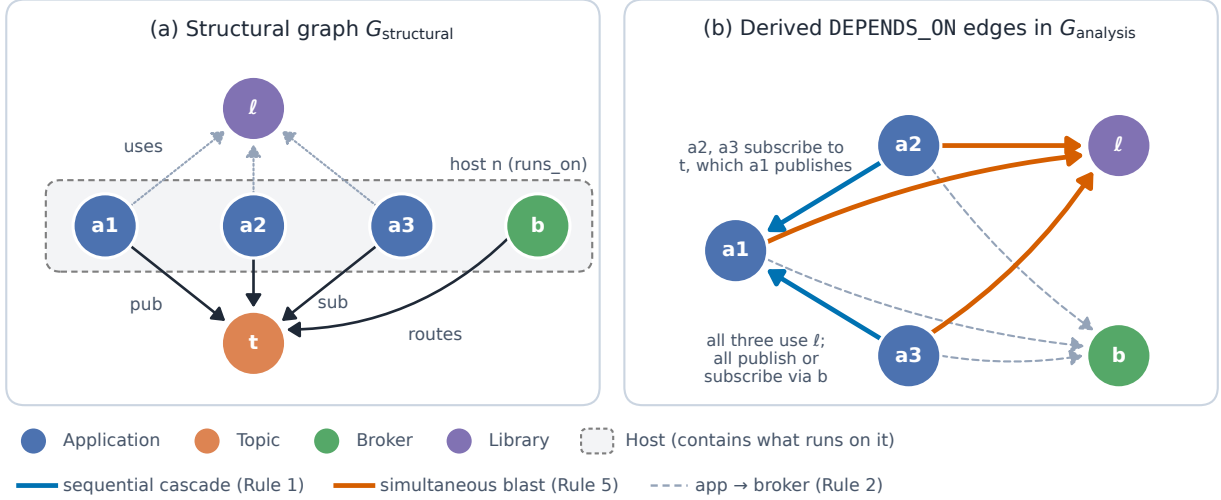


Figure 2: Running example. (a) Three applications share topic t (routed by broker b) and library l , and all run on host n . No structural edge joins two applications. (b) The derived **DEPENDS_ON** edges make the hidden dependencies explicit: subscribers a_2, a_3 depend on publisher a_1 (Rule 1), all applications depend on l (Rule 5), and each application depends on the broker (Rule 2). Simulation oracles run on view (a); the analytical rankings and the $\neg P$ learners read view (b), and the raw-graph learners pass messages over view (a) with node features computed from (b).

3.5. Typed Node Feature Encoding

Both the ranking pathway (Section 4) and the proposed explanation layer use typed node properties from G_{analysis} . All five entity types share a base set of 18 normalized topological metrics (ranging from $[0, 1]$): PageRank, Reverse PageRank, betweenness, closeness, eigenvector centrality, in-degree, out-degree, clustering, articulation, bridge ratio, node QoS weight, incoming and outgoing dependency weights, multi-path coupling, path complexity, fan-out criticality, and the Connectivity Degradation Index (CDI). We calculate four centrality features (PageRank, Reverse PageRank, betweenness, and closeness) over weighted edges, so they include QoS information. This means the so-called QoS-off condition is not entirely free of QoS effects. The in-degree feature and the incoming dependency weight w_{in} (a QoS-weighted version) ensure that all learners receive information closely related to **InDeg**, so the learners inherit some overlap with the oracle’s reference counts (Section 6.2). We did not use any learner missing both columns. Type-specific blocks add more features, extending the vector to 19–25 dimensions (Applications and Libraries add the code-quality penalty and its four static-code inputs; the full schema is in Supplementary Section S12). CDI measures structural connectivity loss using a fixed-scale breadth-first sample to keep computation time reasonable (Section 6.4). We discuss its sensitivity to hash-seeded tie-breaking in Section 7.5.

In this study, SaG therefore serves two roles. It provides a dependency representation from which analytical rankings can be computed directly, and it provides a graph on which learned and hybrid methods operate. The evaluation in Sections 5–7 examines how much ranking performance arises from this representation and how much from the models applied to it.

4. Ranking Methods, Simulation Oracles, and the Reference Criterion

This study evaluates three categories of ranking methods: analytical rankings derived directly from dependency structure, hybrid methods that combine an analytical prior with a learned correction, and learned methods that infer rankings from graph representations. The training-free baseline, **Topo-QoS**, applies QoS-weighted betweenness to the dependency graph, and Section 5.2 defines the analytical references. The learned methods use graph neural networks on the typed multigraph and the dependency graph, and gradient-boosted models with per-Application dependency-graph features (Sections 4.1–4.2 and 5.2). Hybrid methods adjust the **Topo-QoS** score with a learned correction (Figure 3). All methods are trained on or evaluated against simulation oracles that run on a separate graph view (Section 4.3). Because several rankings

are derived from the same failure-propagation assumptions the simulators use, the evaluation distinguishes rankings that restate a simulator’s rule from those that predict its outcomes (Section 4.4). Figure 3 illustrates the methods and their evaluation; full hyperparameters and training commands are in the experiment pages of the replication repository (Section 5.1).

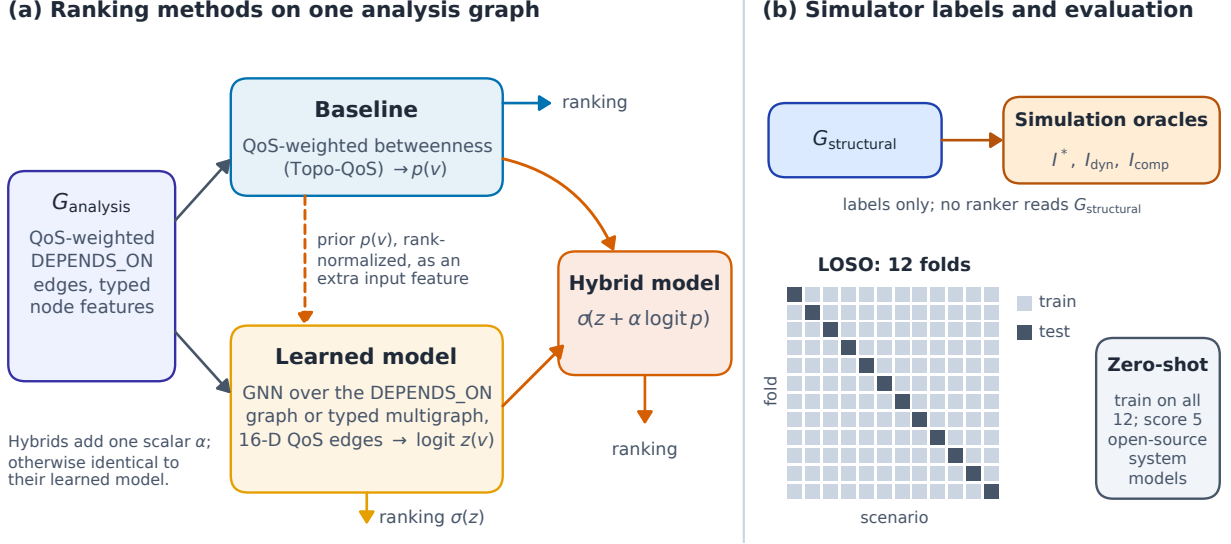


Figure 3: (a) SaG’s ranking methods read the analysis graph. The training-free baseline scores QoS-weighted betweenness; $p(v)$ is that **Topo-QoS** score, rank-normalized to $[0, 1]$ within the graph; the learned model outputs a logit $z(v)$. A hybrid model gives the learned model $p(v)$ as an extra input feature and adds a learned correction to it on the logit scale, $\sigma(z + \alpha \text{logit } p)$, with one learnable scalar α . (b) Labels come from simulation oracles on the structural graph, which no ranker reads. We evaluate the methods by leave-one-scenario-out cross-validation over twelve synthetic architectures and zero-shot on five open-source system models.

4.1. Heterogeneous Graph Transformer and Attention Networks

The main learned ranker, HGT-QoS, is a three-layer Heterogeneous Graph Transformer (HGT) [72] built with PyTorch Geometric [80]. It uses a hidden dimension of $D = 64$ and $H = 4$ heads; on the dependency graph, which has fewer relation types, HGT-P-QoS uses width 100 so that its parameter count (430,680) stays within 1% of HGT-QoS (434,620). Entity-specific linear projections map raw node features $x_v \in \mathbb{R}^{19-25}$ into the hidden space, with $h_v^{(0)} = \text{LayerNorm}(\text{GELU}(W_{\tau(v)}x_v))$. For each meta-relation $\langle \tau(u), \phi(e), \tau(v) \rangle$, attention uses type-parameterized keys, queries, and values, scaled by a learned per-meta-relation prior $\mu_{\langle \tau(u), \phi(e), \tau(v) \rangle}$; full layer-wise formulations are in Supplementary Section S2.1. Message passing runs over both G_{analysis} and its transpose. In the raw multigraph $G_{\text{structural}}$, native relations point away from Applications (e.g., **PUBLISHES_TO**, **RUNS_ON**, **USES**), so forward message passing cannot propagate messages to the scored Application nodes. As a result, forward GNNs on $G_{\text{structural}}$ collapse structurally into per-node MLPs operating over precomputed centrality features (Section 7.2). In contrast, the derived dependency graph G_{analysis} explicitly directs edges from dependents to dependencies, allowing forward message passing to aggregate information from downstream consumers. The homogeneous Graph Attention Network (**GAT-QoS**, and **GAT-P-QoS** on the dependency graph) uses a three-layer architecture with four attention heads and width $D = 288$ ($D = 296$ in the unweighted control for parameter matching), projecting all node types into a joint embedding space before applying homogeneous **GATConv** layers.

Each directed edge has a 16-dimensional vector. Index 0 represents the coupling weight, index 1 is the normalized path count, indices 2 to 8 are a one-hot encoding of the relation, and indices 9 to 15 are middleware QoS parameters such as reliability, durability, priority, transport departure mode, deadline constraints, and max-blocking time. The edge vector is projected and added before attention.

4.2. Prediction Head and Training Objective

A composite head predicts the simulated cascade impact, supplemented by an active auxiliary head $\hat{a}_1$ regressing on the simulator’s reliability column I_R^* . (A secondary maintainability head defined in the schema is masked out of backpropagation because the simulator produces no maintainability labels.) The operational training objective combines mean-squared error regression with listwise and margin-ranking terms:

$$\mathcal{L} = \text{MSE}(\hat{I}^*, I^*) + 0.5 \cdot \text{MSE}(\hat{a}_1, I_R^*) + 0.3 \cdot \mathcal{L}_{\text{rank}} + 0.1 \cdot \mathcal{L}_{\text{pairwise}}, \quad (3)$$

where $\mathcal{L}_{\text{rank}}$ is Listwise Maximum Likelihood Estimation (ListMLE) [81] over the permutation π induced by the simulated labels,

$$\mathcal{L}_{\text{rank}} = -\frac{1}{N} \sum_{i=1}^N \left(\hat{s}_{\pi_i} - \log \sum_{j=i}^N \exp(\hat{s}_{\pi_j}) \right), \quad (4)$$

and $\mathcal{L}_{\text{pairwise}}$ is a margin-ranking loss with $\gamma = 0.05$ over pairs differing by more than γ . Tied labels, which affect about 31% of Applications with $I^* = 0$, enter π in a fixed deterministic sequence (ordered by entity identifier). While this injects an arbitrary ordering among inactive nodes into ListMLE, the margin-ranking term explicitly ignores pairs separated by less than $\gamma = 0.05$, dampening gradient penalties from tied inactive components.

Models are trained with AdamW ($\eta = 3 \times 10^{-4}$, weight decay 10^{-4}) using cosine warm restarts for up to 300 epochs. We set early stopping at 60 epochs of patience on a 20% node-level validation split from the largest training scenario, using five fixed seeds ($\{42, 123, 456, 789, 2024\}$); while this node-level split is transductive with respect to that scenario’s internal graph, all reported evaluations are strictly inductive under leave-one-scenario-out cross-validation. We set hyperparameters and loss coefficients before the first sweep and kept them the same across all experiments. This approach differs from the registered plan, which called for selecting every hyperparameter using nested inner-LOSO selection (two-fold inner holdout). Instead, the published results use one fixed configuration, chosen during the main experiments to keep computation manageable across seeds and folds. We explain this difference in Section 5.3. We ran the registered selection later as a sensitivity analysis on the eight-configuration stage-1 grid (varying depth and rank normalization of features and labels). This analysis also used early stopping on a held-out training scenario, which is another difference from the published protocol (Section 6.2). The results confirm that the main null finding does not change under nested tuning.

4.3. Simulation Oracles

The simulators define three different notions of cascade impact rather than direct observations of real failures, so they serve as evaluation criteria whose assumptions can favor some ranking methods over others. Labels come from three failure simulators, which serve as label oracles (we use the two terms interchangeably), on the raw structural multigraph $G_{\text{structural}}$:

- **Primary reachability cascade oracle (I^*):** Crashes component v , propagates outages through dependent topics, brokers and links via breadth-first traversal on $G_{\text{structural}}$, and computes the mean fractional feed loss across intact subscribers. Feed loss is scaled by a QoS severity ladder ($\times 1.2$ RELIABLE, $\times 1.15$ high priority, $\times 1.05$ medium) and clamped to $[0, 1]$. Each propagation wave is processed in a seeded random order, which decides ties at the propagation threshold; the label is the mean over five seeds. Disabling QoS scaling leaves the Application ordering largely intact ($\rho = 0.965$ across the twelve folds), and substituting durability-aware rescaling moves it less still ($\rho = 0.977$), showing that I^* is predominantly a topological reachability metric.
- **Queue-flow discrete-event oracle (I_{dyn}):** A discrete-event SimPy [82] simulation to model dynamic message emission rates, queue buffer saturation, and network latency. It measures the drop in delivered message rate for surviving consumers under load, using declared rates, payload sizes, and QoS contracts that I^* ignores. QoS policies act on the queues directly: on overflow, a RELIABLE topic drops its oldest message and a BEST_EFFORT topic the newest; durable topics replay retained messages after the fault; transport priority orders service; deadlines and lifespans expire messages; and history depth sets queue

capacity. No run with these policies disabled was made, so the simulator’s sensitivity to QoS is not quantified. Its mechanism differs from reachability, but it shares the same pub-sub topology, and its first-order effect—consumers that lose a failed publisher’s messages—is again subscriber loss. While I_{dyn} models declared rates and buffer limits, it operates on configured parameters free of physical operating-system dynamics like socket buffer backpressure, thread scheduling, or garbage collection pauses. It provides labels for all 1,321 Applications, averaging over five seeds, and takes 12.7 CPU-hours for the full corpus, compared to seconds for I^* . A sensitivity check restricted to the first 30 Applications per fold in lexicographic order is reported in Supplementary Section S38 (Table S62). I_{dyn} agrees with I^* at a mean $\rho = 0.711$ [0.625, 0.794]. Across seeds, a single I_{dyn} run agrees with another at $\rho = 0.43$ –0.96 per fold; the five-seed mean used as the label has an estimated reliability of $r = 0.79$ –0.99 (Spearman–Brown), so no ranker can agree with it by more than $\sqrt{r} \approx 0.89$ –0.996 per fold. Every ranker in Table 6 stays below even r on every fold; per-fold values are in the replication package.

- **Composite multi-criteria failure oracle (I_{comp}):** An auxiliary benchmark uses the Validate-stage failure simulator. We run it on all 1,321 Applications with QoS weighting and a primed baseline flow. Each removal is scored as a linear combination of four degradation components: $I_{\text{comp}}(v) = 0.35 \text{RL}(v) + 0.25 \text{FR}(v) + 0.25 \text{TL}(v) + 0.15 \text{FD}(v)$, representing Reachability Loss (RL), Fragmentation (FR; 0.70 structural component loss + 0.30 severity), Throughput Loss (TL), and Flow Disruption (FD). The weights were chosen, not elicited: the AHP matrix behind them was filled in to reproduce this vector. I_{comp} therefore serves as a declared multi-objective stress test to assess resilience to network fragmentation and throughput disruption, rather than just subscriber loss. Since fragmentation and flow disruption occur with almost every removal, $I_{\text{comp}}(v) > 0$ for nearly every Application. I_{comp} is never used as a training label.

4.4. Input–Label Separation and the Reference Criterion

No simulation output is used as an input: node features are computed on G_{analysis} , and labels come only from the oracles, which execute on $G_{\text{structural}}$; a regression test in the replication package ensures this separation. The separation is between inputs and labels, not between graphs: the raw-graph learners pass messages over the same structural edges the oracles traverse.

Procedural separation does not eliminate construct overlap, which is the main risk discussed in this paper. High agreement with a simulator does not necessarily imply predictive skill: a ranking may correlate strongly with a simulator because it reproduces the simulator’s propagation rule, not because it infers impact from independent evidence.

The reference criterion. We established the formal concept of the reference criterion post hoc to understand why simple dependency counts performed so strongly. A ranking is a *reference* for an oracle O if it is a closed-form simplification of O ’s own computation using the same inputs. A reference measures how much of O comes from its own rule, not predictive skill, so comparisons with it are not claims of superiority. All other rankers, namely the training-free baseline and the learned and hybrid methods, are *predictors*. For I^* , the first propagation wave corresponds to the set of **InDeg** counts (see Remark 1), which is classical publish–subscribe afferent coupling [54, 55]; the references are therefore its first-order expansion (Eq. (6)), **InDeg**, **Reach**, and their raw-graph equivalents. For I_{dyn} , the rate-weighted first-order expansion (Eq. (7)) truncates its computation to direct subscriber loss weighted by declared publication rates, so it is the reference for that simulator; the I^* references are also reported for comparison. For I_{comp} , no ranking is a truncation; its fragmentation and flow terms respond to removing high-degree and high-betweenness nodes, so **Topo-QoS** and raw degree are *term-aligned* with it. They are not references under the definition above, but their performance is interpreted with the same caution. How much of each oracle its own rule recovers is read directly from the reference rows: the first-order expansion reaches $\rho = 0.808$ on I^* , and the rate-weighted expansion 0.830 on I_{dyn} (Tables 5 and 6). The learned methods are not references by this definition, but they use in-degree features and are trained on the oracle they are scored against, so they share some overlap (Section 6.2 measures this). References are therefore reported to show how much agreement can be obtained by encoding a simulator’s assumptions directly, and predictors are evaluated against them to determine whether learning adds information beyond explicit dependency structure. Two analyses limit the circularity instead of removing it. Every ranker is evaluated on all three oracles, and its agreement with I_{dyn} is also

reported after removing the effect of I^* (and separately, the first-order term) (Section 6.1). However, none of these analyses replaces validation against real failures in live distributed systems, which this study does not include (Section 7.5). All results are therefore conditional on simulation fidelity.

5. Experimental Setup

The experimental design evaluates competing explanations of cascade-impact ranking performance. Some rankers derive impact analytically from explicit dependency structure, some combine analytical and learned components, and others learn directly from graph representations. The goal is not only to identify high-performing rankers, but also to determine whether ranking performance originates primarily from the dependency representation or from the learning algorithm operating on it. RQ1 therefore compares learned approaches with analytical rankings derived from the same dependency structure, and RQ2 isolates the contributions of dependency representation, degree information, relation typing, QoS information, and model family.

5.1. Corpus and Replication Package

The corpus includes 2,812 components from seventeen architectures (see Table 3). Twelve synthetic topologies make up the LOSO folds, covering areas like autonomous vehicles, financial trading, healthcare, industrial SCADA, smart-city IoT, telecom RAN, logistics, gaming, microservices, enterprise integration, and air-traffic management. Because all twelve topologies and their code metrics come from the same family, LOSO tests transfer across several parameter settings of that generator, not across unrelated industrial systems. We can regenerate each topology exactly from its configuration, and continuous integration checks this using a SHA-256 manifest. Because every architecture is available both as a raw multigraph and as a derived dependency graph, the corpus separates the effects of dependency representation, analytical structure, and model complexity on ranking performance.

Table 3: Evaluation corpus. The twelve synthetic topologies are the LOSO folds; we exclude the five open-source system models from all training and use them only for zero-shot transfer (Section 6.3). Counts are read from the committed topology files and verified in CI; Supplementary Section S14 gives per-scenario composition.

Dataset	$ V $	$ V_{app} $	Topics	Brokers	Hosts	Libs	$ E $
Synthetic evaluation scenarios (12)	2,461	1,321	615	65	202	258	10,918
Open-source system models (5)	351	141	120	16	32	42	700
Total	2,812	1,462	735	81	234	300	11,618

The five open-source systems are hand-crafted models. Autoware.universe (ROS 2), EdgeX Foundry, Home Assistant, and two meshes based on Online Boutique and Train-Ticket were each created by the first author as typed multigraphs using public architectural documentation. We did not mechanically extract them from source code or container manifests. Because one person modeled them, bias is possible. No second modeler re-derived any model, so this risk is not quantified (Section 7.5). Some details, like brokers, QoS profiles, code metrics, and host specifications, are partly assumed. Two models differ significantly from their originals: the Online Boutique model is a 22-application pub-sub mesh with four brokers, while the original utilizes RPC and has no broker. The Train-Ticket model treats its service-discovery registry as an event broker. In both cases, we changed synchronous RPC interactions to asynchronous request-and-response topics with routing intermediaries (per-system composition in Supplementary Section S14; modeling notes in the replication repository’s RQ3 experiment page). Specifically, synchronous RPC calls were decomposed into pairs of reciprocal topic interactions: a request topic published by the invoking caller and subscribed by the target provider, and a separate reply topic published by the provider and subscribed by the caller, both mediated by broker channels. This explicitly preserves bidirectional dataflow paths while converting synchronous blocking semantics into decoupled event streams, leaving out client-side timeout, thread-pool exhaustion, and synchronous backpressure effects. Neither model includes synchronous call edges. As a result, RQ3 tests zero-shot transfer to stylized architecture models inspired by open-source systems made by a single modeler, not to real production systems or automatically extracted call graphs.

Replication package. All datasets, harnesses, checkpoints, and result artifacts are stored on Zenodo (see Data Availability). In addition, the public repository provides documentation for each experiment, including its protocol, hyperparameters, make target, artifacts, and extended results (<https://github.com/onuralpyigit/software-as-a-graph/tree/jss-submission/docs/research/jss/experiments>).

5.2. Rankers and References

Table 4 groups the rankers into three categories: analytical rankings derived directly from the explicit dependency structure, hybrid rankers that combine an analytical prior with a learned correction, and learned rankers that infer impact from graph representations. This categorization aligns with the study’s objective of comparing analytical, hybrid, and learned explanations of cascade impact under a common evaluation protocol. Other training-free baselines (Degree-raw, PR-raw, RevPR-raw, Topo) are reported in Supplementary Section S40. Analytical rankings that restate a simulator’s propagation rule are not predictors; Table 4 lists them for completeness, and they are described as references below. For GNNs, ‘-QoS’ means using the 16-dimensional QoS edge vector (see Section 4.1) along with three QoS node columns (w , w_{in} , w_{out}). ‘Hybrid-X’ refers to model X adjusted by the baseline prior. GAT and GAT-QoS are untyped GATs with the same parameter budget as HGT, and GAT-QoS uses the same 16-D edge vector as HGT-QoS. Together, these define a 2x2 grid based on typing (GAT vs. HGT) and the QoS channel. The study also tested smaller and projection-based variants, which are listed in Supplementary Section S32. The learned and hybrid models use the native typed multigraph under LOSO, while the ‘-P’ models use the Application–Library `DEPENDS_ON` graph, with identical graph node features and labels. In QoS-off settings, all edge weights are set to 1, and the QoS node columns are set to zero, but four centralities still use QoS (see Section 3.5).

Table 4: Rankers and references reported in the main text, grouped as analytical, hybrid, and learned. **-QoS**: QoS-weighted distances (**Topo-QoS**) or the 16-D QoS edge vector plus three QoS node columns (GNNs); **-P**: trained on the derived dependency graph; **Hybrid-X**: model X corrected by the **Topo-QoS** prior; **→dyn**: trained on I_{dyn} labels. Unsuffixed GNNs (GAT, HGT) serve as raw-multigraph confound controls ($G_{\text{structural}}$). Params: trainable parameters (— for tree ensembles and training-free rankings). References restate a simulator’s rule and carry no contrast against **Topo-QoS** (Section 4.4); ablation arms (**GIN-P-QoS**, Table 8; **GBM-Feat**) and the Amendment 16 arms (**GAT-QoS-R**, the corrected- and **InDeg**-prior hybrids; Table 9) are reported in Section 6.2. [¶]**Topo-QoS** carries an implementation defect where the articulation point term is zero due to an index mismatch (so it acts as pure QoS-weighted betweenness, scoring 0.553; restoring the term lowers the score to 0.533, confirming that the defect does not disadvantage the baseline).

Ranker	Graph read	Typing / edge input	Params
<i>Analytical: training-free baseline (registered comparator)</i>			
Topo-QoS[¶]	App-Lib <code>DEPENDS_ON</code>	scalar $w(e)$; betweenness only (AP term zero; corr. 0.533)	—
<i>Analytical: references (no contrast; Section 4.4)</i>			
Analytic I^* (Eq. (6))	topic publish/subscribe sets	first-order subscriber loss	—
Rate-weighted (Eq. (7))	topic publish/subscribe sets	first-order loss weighted by declared rates	—
InDeg / Reach	App-Lib <code>DEPENDS_ON</code>	direct / transitive dependents	—
<i>Hybrid: learned correction of the Topo-QoS prior</i>			
Hybrid-GAT / Hybrid-HGT	$G_{\text{structural}}$ + features	as base + Topo-QoS prior	431,433 / 434,941
<i>Learned: dependency graph</i>			
GAT-P-QoS, HGT-P-QoS	App-Lib <code>DEPENDS_ON</code>	homogeneous / heterogeneous	429,992 / 430,680
<i>Learned: raw multigraph, typing × QoS channel at matched budget</i>			
GAT / GAT-QoS	$G_{\text{structural}}$ + features	homogeneous; none / 16-D	437,496 / 429,992
HGT / HGT-QoS	$G_{\text{structural}}$ + features	heterogeneous; 1-hot / 16-D	434,620
<i>Learned approximations of I_{dyn} (trained on I_{dyn} labels)</i>			
GBM-P-QoS→dyn	per-Application features (App-Lib <code>DEPENDS_ON</code>)	18 columns: 9 dependency counts, 9 QoS/rate	—
GAT-P-QoS→dyn	App-Lib <code>DEPENDS_ON</code>	homogeneous; 16-D	429,992

Training-free baseline. **Topo-QoS** is computed on the Application–Library `DEPENDS_ON` graph (Rules 1 and 5):

$$\text{Topo-QoS}(v) = 0.6 \cdot \text{BT}_w(v) + 0.4 \cdot \text{AP}(v), \quad (5)$$

where BT_w is betweenness over edge distances $d(e) = 1/(w(e) + 10^{-6})$, so tightly coupled edges attract shortest paths, and AP flags articulation points. Because of an implementation defect (an index mismatch when the articulation scores are read), the AP term is zero for every node, so the score that was executed, and that every table reports, is QoS-weighted betweenness alone. Restoring the term lowers **Topo-QoS** from 0.553 to 0.533, so the defect does not favor the reference counts. The hybrids’ prior $p(v)$ inherits the same defect;

they were not retrained with the corrected prior. The unweighted Topo baseline is reported in Supplementary Section S40.

Because several analytical rankings are derived from the same architectural information the simulators use, high correlation with a simulator does not by itself imply prediction (Section 4.4). These analytical approximations serve as strong references because they are derived directly from the corresponding failure-propagation assumptions. Three primary training-free rankings restate the reachability oracle’s propagation rule and are reported as references. The first is the post hoc first-order expansion of I^* (Analytic I^*):

$$\hat{I}_1^*(v) = \sum_{t \in \text{pub}(v)} \frac{|\text{sub}(t)|}{|\text{pub}(t)|}, \quad (6)$$

where $|\text{pub}(t)| \geq 1$ holds for all published topics $t \in \text{pub}(v)$ by construction (v publishes to t), preventing any division by zero. The second is $\text{InDeg}(v)$, the number of direct dependents of v on the Application–Library `DEPENDS_ON` graph, which for an Application equals publish–subscribe afferent coupling and is exactly the set I^* ’s first wave reaches (Remark 1). The third is $\text{Reach}(v)$, the number of its transitive dependents normalized by $|V| - 1$, which is the set a reachability cascade can visit. For the queue-flow simulator I_{dyn} , the corresponding reference is the rate-weighted first-order expansion

$$\hat{I}_{\text{dyn},1}^{\text{rate}}(v) = \sum_{t \in \text{pub}(v)} \frac{r_t}{|\text{pub}(t)|} |\text{sub}(t)|, \quad (7)$$

where r_t is the declared publication frequency of topic t . Eq. (7) weights each term of the first-order expansion of I^* (Eq. (6)) by the topic’s declared publication rate. It is therefore a first-order truncation of the queue-flow simulator I_{dyn} , which reads the same rates, and it is reported as a reference for I_{dyn} rather than as a predictor. It was evaluated after the registered analyses and is exploratory. Because it incorporates the same rate information used by I_{dyn} , it is the natural reference against which learned approximations of that simulator are evaluated; in practice, it can also serve as a training-free ranking for queue-flow impact (Section 7.4). In addition, we archive raw-multigraph topic counts (Pubs-raw) and transitive closure without Rule 5 (Reach-R1) in the replication repository. A reference shows how much of an oracle a restatement of its rule recovers, which is the size of the circularity, not predictive skill. References carry no contrast against **Topo-QoS**, and a learner’s distance from them is reported as such, not as a test. The one exception is the paired comparison between the rate-weighted reference and the learned queue-flow approximation (Section 6.1), which is labeled exploratory.

5.3. Metrics, Protocols and Statistics

Population. Every ranker is scored on the Application set V_{app} .

Metrics. We use Spearman ρ to measure ranking against the simulation oracles. To distinguish between active components and those with zero impact, we report both the full-population Spearman ρ and the active-stratum Spearman $\rho_{>0}$, which only includes components with true impact greater than zero. To identify critical sets, we use $\text{Overlap}@K$, the fraction of the true top- K recovered by the predicted top- K , where K is about 20% of $|V_{\text{app}}|$. PR-AUC, $F_1@7$, and nDCG@10 are reported in Supplementary Section S16.

Protocols. In the LOSO setup, we train learned models on eleven scenarios and test on the twelfth, repeating this for all 12 folds and five random seeds. The reported value is the average ρ across seeds. For zero-shot transfer, we evaluate models built on all twelve scenarios across the five system models without any fine-tuning. Training-free rankers are not fitted to any data, so for them, both protocols become per-scenario analyses on the same twelve scenarios and five models. We include them in the same tables to ensure every row is scored on the same scenarios and labels.

Statistics. The analysis emphasizes effect sizes with confidence intervals and relative ranking quality, rather than identifying a single highest-performing model. Throughout, “matches” means that two rankers are not significantly different; it does not imply statistical equivalence, which is not established for any pair at ± 0.05 (Section 7.5). The summary ρ is the average Spearman correlation for each fold or system, with a 95% confidence interval from a bootstrap that resamples folds ($B = 2,000$) [83]. We compare results using

paired two-sided Wilcoxon signed-rank tests across folds [84], following guidance on statistical testing in empirical software engineering [85]. With 12 folds, the smallest possible two-sided p -value is 0.00049. Fisher- z averaging does not change the row order in Table 5. Weighting folds by $|V_{\text{app}}|$ keeps the top five the same (the reference **InDeg** rises to 0.810) but increases **Topo-QoS** to 0.596, corresponding to the raw-multigraph learned rankers (0.595–0.606). Spearman ρ gives tied scores their average rank. $\text{Overlap}@K$ breaks ties at the K boundary by node order, which is arbitrary for integer counts, so Section 6.1 also reports tie-aware recall curves. Because LOSO folds share ten of eleven training scenarios, we also test every registered contrast with the Nadeau–Bengio corrected resampled t -test [86] and, when weighting by $|V_{\text{app}}|$, with an exact sign-flip permutation test.

Analysis plan and the status of each result. The evaluation protocol is version-controlled in the replication repository. We registered sixteen numbered amendments across the study, encompassing sensitivity, ablation, and reclassification arms, with none withheld. We assign every reported result to one of three tiers. *Confirmatory*: the original plan’s two co-primary contrasts, QoS-weighted (HGT-QoS vs. Topo-QoS) and unweighted (HGT vs. Topo-QoS), Holm-corrected across the two; both were null. *Registered secondary*: hypotheses committed before their respective arms executed—including the matched 2×2 , hybrid rankers, reference counts, dependency-graph learners, derivation value, full-population queue-flow labels, sensitivity arms, and the direction and corrected-prior arms of Amendment 16, each Holm-corrected within its family. We conducted all secondary analyses after observing the primary null. *Exploratory*: post hoc investigations, reference reclassifications (which altered no figures; Section 4.4), and the rate-weighted reference for I_{dyn} (Eq. (7)) with the input attribution of the learned queue-flow approximation, both added after all other results existed. **Plan deviation**: the registered nested inner selection was simplified to a fixed configuration during primary sweeps; a sensitivity analysis assesses the registered selection (Section 6.2). Because sequential amendment registration accumulates family-wise error throughout rounds, we also evaluate an omnibus Holm correction across all 13 decision-bearing contrasts. Both hybrid contrasts against Topo-QoS remain significant under this omnibus adjustment (Hybrid-GAT $p_{\text{omni}} = 0.019$, Hybrid-HGT $p_{\text{omni}} = 0.041$). In contrast, the primary contrast remains null (+0.069, nominal $p = 0.266$, $p_{\text{omni}} \geq 0.46$). Main-text tables report per-family Holm corrections to reflect the pre-specified hypothesis families; the amendment log in Supplementary Section S26 lists each family and its contrasts. The headline findings of this paper are therefore registered secondary or exploratory, not confirmatory.

Taken together, the rankers allow comparison of representation-centric and model-centric explanations of ranking performance. If performance arises primarily from the dependency representation, analytical and learned rankers built on that representation should perform similarly. If learning adds information, learned rankers should outperform analytical alternatives built on the same dependencies. Section 6 reports the results against these two expectations.

6. Results

The evaluation asks not only which model ranks best but also where that ranking performance comes from. We therefore read the results along three dimensions: the contribution of explicit dependency representations, the relative performance of analytical, hybrid, and learned approaches, and the distinction between rankings that restate a simulator’s rule and those that predict its outcomes.

All results are reported on the Application population (V_{app}) against the primary oracle $I^*(v)$, and, for the rankers of Table 6, also against the queue-flow oracle $I_{\text{dyn}}(v)$ and the multi-criteria oracle $I_{\text{comp}}(v)$ (Section 4.3). Only the two co-primary contrasts are confirmatory; every other contrast is registered secondary or exploratory (Section 5.3). Per-fold results, secondary strata, and extended protocol notes are in the public experiment pages (Section 5.1). Figure 4 summarizes the main findings.

6.1. RQ1: Ranking Accuracy

Summary. For the reachability and queue-flow oracles, explicit dependency graphs, rather than model complexity, carry most of the ranking signal. The strongest learned model, the graph attention network on the dependency graph (GAT-P-QoS, $\rho = 0.748$), approaches without exceeding the direct-dependent reference

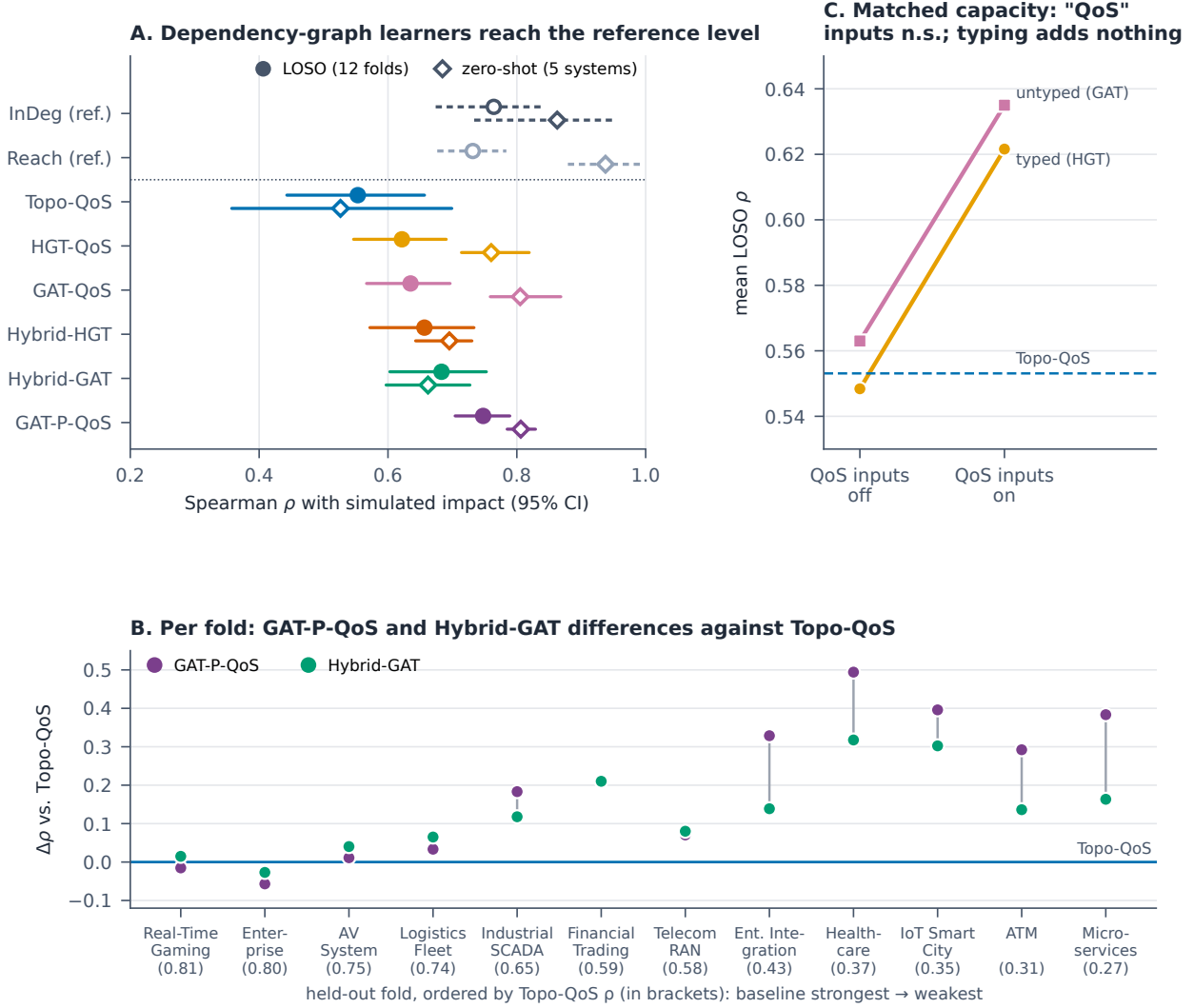


Figure 4: Main results for the Application population. (A) Shows the mean Spearman ρ with 95% bootstrap intervals under LOSO (filled circles; see Table 5) and zero-shot results on five system models (open diamonds). Grey hollow markers represent reference rankings that restate I^* 's propagation rule (InDeg, Reach) and are not predictors. The neural model using the dependency graph (GAT-P-QoS) is not statistically distinguishable from InDeg, though equivalence within ± 0.05 is not established (TOST $p = 0.17$). (B) Shows the improvement of GAT-P-QoS and Hybrid-GAT over Topo-QoS for each held-out fold. (C) Displays cell means from the capacity- and channel-matched 2×2 (Table 7): the "QoS" inputs, which include a QoS-weighted in-degree column, increase both models by about 0.07 (not significant after Holm correction), while typed and untyped configurations show no difference. When w_{in} is held in both arms, the "QoS" effect drops to +0.030 (Table 8).

(0.764; not statistically distinguishable, but equivalence at ± 0.05 not established), and ablations show that it draws on its degree features (losing 0.14 without them; see Section 6.2). For the more expensive queue-flow oracle I_{dyn} (12.7 CPU-hours), a learned approximation trained on its labels reaches $\rho = 0.799$, above the unweighted first-order expansion (0.706, Holm $p = 0.019$) because it reads declared publication rates and payload sizes; the rate-weighted reference (Eq. (7)), which reads the same rates without training, reaches 0.830 and exceeds the learned approximation (+0.031, 10 of 12 folds; exploratory). Under multi-criteria failure conditions (I_{comp}), hybrid rankers remain stable (0.58) while pure neural models drop sharply (0.144–0.334), and training-free scores aligned with the oracle’s terms rank higher still. On the raw multigraph, the main registered comparison (HGT-QoS vs. Topo-QoS) showed no significant difference ($\Delta\rho = +0.069$, $p = 0.266$). To recover 80% of the true top-20% set, GAT-P-QoS must flag the top 40%.

Learned rankers are trained on eleven scenarios and scored on the twelfth; training-free rankers are scored on all twelve (26 to 300 Applications, K between 5 and 60). Table 5 reports all of them against I^* , with the reference rankings of Section 5.2 in a separate block: they restate I^* ’s rule, so their rows measure the circularity and carry no contrast.

Two patterns emerge. First, reading the dependency graph instead of the raw multigraph improves attention-based learners (GAT-P-QoS 0.748 vs. GAT-QoS 0.635), although the typed transformer becomes unstable on it (HGT-P-QoS 0.514). Second, analytical rankings derived from the same dependency structure are not exceeded by any learned model (0.764–0.808, against at most 0.748). The analyses below examine both observations.

Table 5: Main results under LOSO across twelve synthetic architectures (Application population; learned rankers: five seeds, CPU sweeps). $\Delta\rho$ is paired by fold against Topo-QoS, with a bootstrap 95% CI ($B = 2,000$) and a two-sided Wilcoxon signed-rank test. $\rho_{>0}$ denotes Spearman correlation restricted to active components ($I^* > 0$). p_{Holm} is given within each registered family: hybrids (registered secondary). The reference block holds canonical rankings that restate I^* ’s propagation rule (the post hoc first-order expansion, Eq. (6); the dependency counts); they are not predictors and carry no contrast. [‡]Exploratory: contrasts against the references and raw counterparts. Note: Topo-QoS carries an implementation defect (articulation term zero; Section 5.2); corrected value is 0.533. Additional training-free baselines and exploratory raw-graph references (Pubs-raw, Reach-R1, Degree-raw, RevPR-raw, Topo), per-fold values, contrasts among references, and GAT-P+InDeg are provided in Supplementary Section S40 and the replication repository. Underlined: best predictor per column.

Ranker	LOSO ρ [95% CI]	Active $\rho_{>0}$	$\Delta\rho$ vs Topo-QoS [95% CI]	Won	p (p_{Holm})	Overlap@K
<i>Reference: restatements of I^*’s propagation rule (not predictors)</i>						
Analytic I^*	0.808 [0.755, 0.853]	0.631	—	—	—	0.536
InDeg (direct dependents)	0.764 [0.674, 0.840]	0.516	—	—	—	0.504
Reach (transitive dependents)	0.732 [0.674, 0.782]	0.286	—	—	—	0.344
<i>Training-free baseline (registered comparator)</i>						
Topo-QoS	0.553 [0.443, 0.657]	0.280	—	—	—	0.388
<i>Learned, on the dependency graph</i>						
GAT-P-QoS	<u>0.748</u> [0.704, 0.789]	<u>0.440</u>	+0.195 [+0.100, +0.294]	10/12	0.0068 [‡]	<u>0.454</u>
HGT-P-QoS	0.514 [0.392, 0.636]	0.237	−0.039 [−0.155, +0.077]	4/12	0.622 [‡]	0.380
<i>Learned and hybrid, on the raw multigraph</i>						
HGT-QoS	0.622 [0.547, 0.690]	0.312	+0.069 [−0.046, +0.174]	8/12	0.266	0.426
GAT-QoS	0.635 [0.567, 0.696]	0.338	+0.082 [−0.046, +0.201]	7/12	0.233	0.438
Hybrid-HGT	0.657 [0.572, 0.733]	0.345	+0.103 [+0.055, +0.152]	11/12	0.0034 (0.0068)	0.435
Hybrid-GAT	0.683 [0.603, 0.753]	0.362	+0.130 [+0.075, +0.190]	11/12	0.0015 (0.0029)	0.450

The reference level. As noted in Remark 1, InDeg represents publish–subscribe afferent coupling [54, 55, 56] and counts exactly the Applications reached by the first propagation wave of I^* . Reach counts the transitive dependents that a reachability cascade can visit. Their values on I^* (0.764, $\rho_{>0} = 0.516$; 0.732) show how much of the oracle is recovered by expressing again its rule, rather than measuring predictive skill. Exploratory raw-multigraph references are evaluated in the replication repository: counting published topics (Pubs-raw) achieves 0.731, and transitive closure without Rule 5 (Reach-R1) gets 0.674. Among the references, that derivation matters in one case: without Rule 5, Reach drops by 0.058 (9/12 folds, Holm $p = 0.0068$; derivation ablation). Untyped raw-graph metrics do not recover the reference: total degree reaches 0.199 and reverse PageRank 0.089, while PageRank is constant for every Application because they have no incoming raw edges.

The first-order expansion of the oracle. The strongest reference is the post hoc closed-form first-order expansion of I^* (Eq. (6), Section 5.2). It reaches mean $\rho = 0.808$ [0.755, 0.853] ($\rho_{>0} = 0.631$): direct

subscriber loss is most of what I^* measures, and InDeg is a coarser version of the same quantity. As a reference, it bounds the mean rather than every fold: on four folds (AV, Financial Trading, Healthcare, Industrial SCADA), InDeg slightly exceeds it (by 0.005–0.038).

Learners on the dependency graph approach the reference. Trained on the dependency graph rather than the raw multigraph, the strongest learned model, GAT-P-QoS, achieves $\rho = 0.748$, approaching but not exceeding the direct-dependent reference. This is 0.016 below the direct-dependent count for individual seeds, and the five-seed ensemble is 0.007 above it (0.772, Table 6). These differences are not statistically significant, nor do they show equivalence: the 90% interval of the per-fold difference is ± 0.076 , so a two one-sided test at the registered margin of ± 0.05 does not pass ($p = 0.17$ per seed, 0.13 for the ensemble). At this sample size, the learner and the count are indistinguishable and not ranked. The closeness to the reference depends on the degree features each learner receives (Section 3.5): removing `in_degree` and w_{in} lowers GAT-P-QoS to 0.612 (Section 6.2). We archive a version that uses InDeg as its prior in the replication repository, since it is a reference prior. HGT-P-QoS only reaches 0.514 and is highly dependent on the seed (mean within-fold seed SD 0.254; Section 7.5).

Hybrid rankers versus the training-free baseline and their base learners. Hybrid-HGT achieves $\rho = 0.657$ (+0.103, Holm $p = 0.0068$) and Hybrid-GAT reaches 0.683 (+0.130, Holm $p = 0.0029$) compared to Topo-QoS, each outperforming the baseline in 11 out of 12 folds. Both results remain statistically significant under the omnibus Holm correction ($p_{\text{omni}} = 0.041$ and 0.019), the Nadeau–Bengio corrected t ($p = 0.019$ and 0.014), and $|V_{\text{app}}|$ -weighted averaging (exact sign-flip $p = 0.038$ and 0.005). In this comparison, a hybrid uses $p(v)$ as an input and adds a learned correction to its logit, nesting its comparator. When compared to their own base learners, neither hybrid shows a significant difference (Hybrid-HGT vs. HGT-QoS +0.035, $p = 0.73$; Hybrid-GAT vs. GAT-QoS +0.048, $p = 0.30$). Retrained with the articulation defect corrected (Amendment 16; Table 9), both hybrids still beat the corrected baseline (0.533): Hybrid-GAT-AP reaches 0.669 (+0.136, Holm $p = 0.0059$) and Hybrid-HGT-AP 0.640 (+0.107, Holm $p = 0.0073$), each on 11 of 12 folds, and again neither differs from its base learner (+0.034 and +0.018, Holm $p = 0.94$). The hybrid gain therefore belongs to the weak comparator, not to the learned correction. Given the direct-dependent count as their prior instead, GAT-QoS+InDeg (0.763) and HGT-QoS+InDeg (0.759) gain +0.128 and +0.137 over their base learners (Holm $p = 0.0049$) and land within ± 0.012 of InDeg itself (equivalent at ± 0.05 , TOST $p < 0.001$): a learner handed the reference reproduces it. The prior also reduces transfer performance: in zero-shot tests, both hybrids perform worse than their base learners (Section 6.3). On their own, the raw-multigraph learned rankers do not differ significantly from Topo-QoS (HGT-QoS +0.069, $p = 0.266$; GAT-QoS +0.082, $p = 0.233$).

Beyond the reachability oracle. Table 6 compares the rankers against two other oracles. On the full-population I_{dyn} , which matches I^* at $\rho = 0.711$ [0.625, 0.794], the reference methods outperform all GNNs (first-order expansion 0.706, InDeg 0.664, compared to 0.496–0.615 for GNNs), and Topo-QoS achieves 0.471. Since I_{dyn} shares its topology and first-order effect with I^* , the partial correlation shows how much of each ranking remains after removing the oracle it was fitted to or restates. After removing the rank of I^* , InDeg keeps 0.272 [0.188, 0.366], Reach 0.117, Topo-QoS 0.134, and the GNNs 0.107–0.173, with all intervals excluding zero. If we remove the first-order expansion instead, the order reverses: InDeg stays at 0.079. Meanwhile, the raw-multigraph GNNs retain most of their value (0.212–0.217), indicating that the learners capture queue-flow signals beyond the unweighted first-order term, although less than the rate-weighted reference (0.573). The queue-flow oracle separates what the dependency structure provides from what declared rates add. The unweighted first-order expansion (Eq. (6)) reaches 0.706 on I_{dyn} . Weighting each of its terms by the topic’s declared publication rate (Eq. (7)) raises it to 0.830 [0.778, 0.872] without training (exploratory). A gradient-boosted model trained on I_{dyn} labels, using counts, the first-order expansion, and the registered QoS feature block, which also carries each Application’s declared publication rate and payload size, reaches 0.799. It outperforms the unweighted expansion (+0.094, Holm $p = 0.019$) and InDeg (+0.135, Holm $p = 0.019$), but falls below the rate-weighted reference, which exceeds it by +0.031 [+0.013, +0.049] on 10 of 12 folds (nominal Holm $p = 0.009$; exploratory). The feature block provides the gain over the unweighted expansion (+0.095 over the same model without it, Holm $p = 0.014$), while on I^* it adds nothing (−0.003, $p = 0.62$). An exploratory attribution splits the block (Supplementary Section S39): the declared rate and payload columns alone add +0.097 (9/12 folds, Holm $p = 0.021$), and the seven QoS-derived columns

Table 6: Rankers against three oracles, twelve LOSO folds, Application population (registered secondary and exploratory). All three oracles are exhaustive over the 1,321 Applications; I_{dyn} is the five-seed mean across all components. Partial ρ : Spearman correlation with I_{dyn} after the rank of I^* , or of the first-order expansion $\hat{I}_1^*$ (Eq. (6)), is regressed out of both. Published learned rows score the mean of the five seeds’ predictions, which raises ρ on I^* above the per-seed means of Table 5; the learned approximations of I_{dyn} are per-seed means. *Trained on I_{dyn} labels of the eleven training scenarios: a learned approximation of I_{dyn} , not an independent check of it. Reference rows restate a simulator’s rule (Section 4.4): the first block restates I^* ’s, and Eq. (7) I_{dyn} ’s. (For **Topo-QoS**, the corrected I^* value is 0.533; Section 5.2.) Underlined: highest value per column among predictors (for I^* , **GAT-P-QoS** achieves 0.772 as a 5-seed prediction ensemble; among per-seed evaluations, **GBM-P-QoS**→**dyn*** achieves 0.764). 95% CIs are bootstrapped across folds.

Ranker	I^* ρ	I_{dyn} ρ [95% CI]	Partial $\rho(\cdot, I_{\text{dyn}} I^*)$ [95% CI]	Partial $\rho(\cdot, I_{\text{dyn}} \hat{I}_1^*)$	I_{comp} ρ
<i>Reference: restatements of I^*’s propagation rule (not predictors)</i>					
Analytic I^*	0.808	0.706 [0.629, 0.783]	0.318 [0.234, 0.412]	—	0.636
InDeg	0.764	0.664 [0.562, 0.758]	0.272 [0.188, 0.366]	0.079	0.650
Reach	0.732	0.583 [0.510, 0.651]	0.117 [0.055, 0.184]	0.102	0.302
<i>Reference: restatement of I_{dyn}’s first-order rule (not a predictor; exploratory)</i>					
Rate-weighted (Eq. (7))	0.756	0.830 [0.778, 0.872]	0.578 [0.471, 0.679]	0.573	0.551
<i>Training-free baseline</i>					
Topo-QoS	0.553	0.471 [0.381, 0.559]	0.134 [0.061, 0.204]	−0.091	<u>0.702</u>
<i>Learned GNNs (seed ensemble)</i>					
GAT-P-QoS	<u>0.772</u>	0.615 [0.549, 0.673]	0.111 [0.016, 0.213]	0.182	0.274
Hybrid-GAT	0.702	0.599 [0.514, 0.669]	<u>0.173</u> [0.082, 0.267]	0.056	0.585
Hybrid-HGT	0.672	0.573 [0.497, 0.638]	0.162 [0.074, 0.252]	0.029	0.582
HGT-QoS	0.667	0.549 [0.464, 0.622]	0.130 [0.035, 0.224]	0.212	0.201
GAT-QoS	0.647	0.523 [0.436, 0.606]	0.107 [0.018, 0.199]	0.217	0.144
HGT-P-QoS	0.618	0.496 [0.395, 0.601]	0.112 [0.033, 0.190]	0.087	0.334
<i>Learned approximations of I_{dyn} (trained on its labels)</i>					
GBM-P-QoS → dyn *	0.764	<u>0.799</u>	—	—	0.477
GAT-P-QoS → dyn *	0.734	0.598	—	—	0.368

($w(t)$ -weighted scores and policy shares) add -0.005 ($p = 0.68$). Declared rates and payload sizes, rather than QoS policies, provide most of the additional information on this oracle, and the analytical formula uses that information at least as effectively as the learned model. The GNN trained on the same labels approaches neither: **GAT-P-QoS**→**dyn** reaches 0.598, below the first-order expansion (-0.108 , Holm $p = 0.014$) and the gradient-boosted approximation (-0.201 , Holm $p = 0.0015$), and is no better than the same GNN trained on I^* (-0.008); unlike the tree model, it received neither rates nor payloads (Section 7.2). The learned approximations are trained on the oracle they are evaluated on, so their possible value is cost, since labeling I_{dyn} took 12.7 CPU-hours for the corpus (Section 6.4); the rate-weighted reference offers the same saving without training. When trained on I^* instead, the same gradient-boosted model is less robust across the three oracles than the first-order expansion alone (worst-case $\rho -0.121$, Holm $p = 0.001$).

The multi-criteria oracle favors different rankers. On I_{comp} , which includes fragmentation, throughput, and flow disruption (Section 4.3), learned rankers without a prior perform poorly (0.144–0.334; sum-aggregation GNNs from Section 6.2 reach 0.41–0.44). The two hybrids that use **Topo-QoS** as a prior maintain 0.58. Training-free scores that correspond to the oracle’s terms rank higher: **Topo-QoS** achieves 0.702 [0.649, 0.753] (higher than the direct-dependent reference at 0.650; the reference is ahead on only 3 of 12 folds, $p = 0.110$), and raw total degree reaches 0.719. **Reach** drops to 0.302. This lead reflects overlap with I_{comp} ’s fragmentation and flow terms, similar to the references’ lead on I^* (Section 4.4), rather than predictive skill. No ranker is best across all three oracles.

Identifying the critical set. Rank correlation across all Applications is not the main concern in a CI gate; the key question is whether the true top-ranked items are flagged. Figure 5 shows, for each ranker, the proportion of the true top-20% set on I^* that appears within its top $k\%$, with ties resolved by averaging. At $k = 20\%$, **GAT-P-QoS** recovers 0.48 of the critical set and **Topo-QoS** recovers 0.38, matching their $\text{Overlap}@K$ (0.454 and 0.388). This means about half of what a top-20% gate flags is not in the true top 20%, and about half of the true top 20% is missed. To recover 80% of the critical set, **GAT-P-QoS** must flag the top 40% (0.81), and **Topo-QoS** does not reach 80% within 50%. The reference methods do only slightly better:

at $k = 20\%$, **InDeg** recovers 0.49 (tie-breaking bounds 0.48–0.50), and 80% recall requires the top 45% by **InDeg** (0.83) or by the first-order expansion (0.89). Only the first-order expansion reaches 90% within the top 50%, and **Reach** does not reach 80% within 50%. On the exhaustive I_{dyn} labels (right panel of Figure 5), **GAT-P-QoS** reaches 0.74 at 40% and 0.82 at 50%, compared to 0.80 at 40% for the **InDeg** reference. The rate-weighted reference (Eq. (7)), which restates I_{dyn} ’s first-order rule, recovers 0.81 of that set at 20% and 0.96 at 30% (exploratory). In practice, requiring a gate to flag 40%–50% of components to catch 80% of critical points would likely cause alert fatigue in production CI/CD pipelines. This suggests that SaG is best operationalized as a prioritized informational triage list for architectural reviews, rather than as an automated blocking gate (Section 7.4 proposes a two-tiered triage protocol).

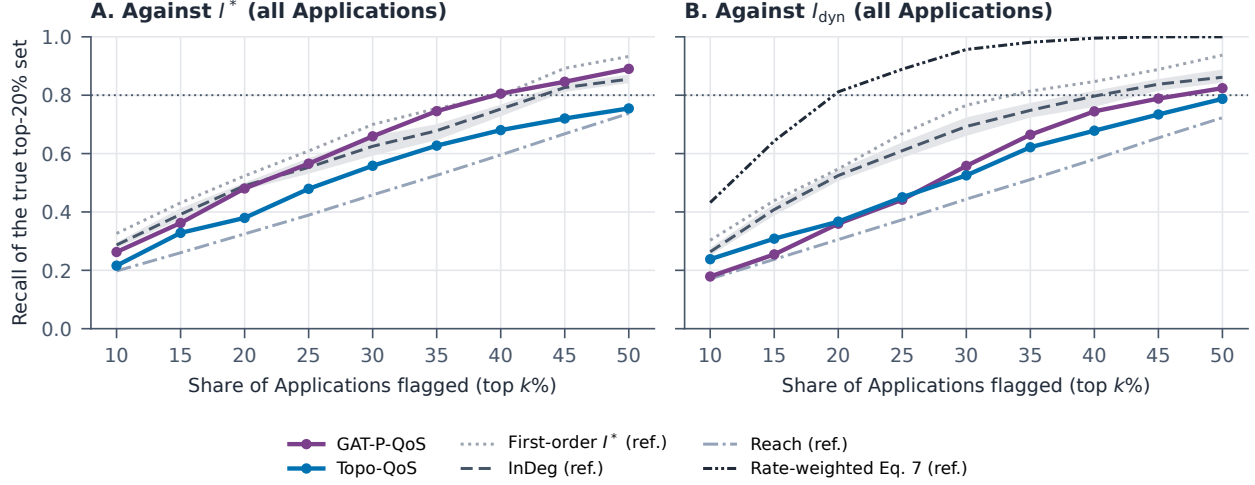


Figure 5: Recall of the true top-20% set (tie-inclusive) by each ranker’s top $k\%$, mean over the twelve LOSO folds, against I^* (left) and I_{dyn} (right), both over all 1,321 Applications, the population of Table 6. Ties are resolved in expectation over random tie-breaking; grey dashed curves are reference rankings that restate I^* ’s rule, and the black dash-dotted curve in panel B is the rate-weighted expansion (Eq. (7)), the reference for I_{dyn} ; the shaded band gives **InDeg**’s optimistic and pessimistic bounds. **Reach**, whose scores tie heavily, has much wider bounds (0.12–0.80 at $k = 20\%$ on I^*), so its expected curve is the only meaningful summary. The dotted line marks 80% recall.

6.2. RQ2: Sources of Predictive Performance

Summary. Attention-based learners improve when using the dependency graph, where messages reach the scored Applications from their dependents (**GAT-P-QoS** 0.748 vs. **GAT-QoS** 0.635), whereas the typed transformer does not in the tested configuration (**HGT-P-QoS** 0.514 vs. **HGT-QoS** 0.622). The dependency-graph GAT’s closeness to the reference comes from its degree features: without them, it loses 0.14 (Holm $p = 0.005$). A sum-aggregation GNN, which can count dependents, loses only 0.01 without these features, though the difference is not significant after correction. When capacity is matched, relation typing has no effect; with weighted in-degree included in both arms, the “QoS” inputs also add no measurable benefit (+0.030, Holm $p = 0.33$). Across these ablations, the largest drops come from removing degree information (−0.136 for **GAT-P-QoS** and −0.270 for **GAT-QoS**), whereas relation typing (−0.013) and the choice of aggregator (−0.016) have little effect. For attention-based learners, the gain from the dependency representation (+0.113) exceeds every change of model architecture tested and survives a control that passes raw-graph edges in both directions (+0.072 over it, Holm $p = 0.014$), suggesting that most of the recoverable signal lies in explicit dependency structure rather than in higher-order message passing.

The “QoS” factor was mostly the weighted in-degree. In Table 7, the main effect of QoS (+0.073) is not significant after Holm correction ($p_{\text{Holm}} = 0.127$), and it cannot be attributed to QoS content. w_{in} , the sum of incoming dependency weights, is a QoS-weighted version of **InDeg**, and the QoS-off arms set it to zero. A sensitivity analysis repeated the matched 2×2 with w_{in} included in the QoS-off cells (Table 8, F4): the main effect drops to +0.030 [−0.023, +0.085] (Holm $p = 0.33$), typing is −0.036, and the interaction is +0.046, none of which are significant. This means more than half of the QoS effect in Table 7 was due to

Table 7: The 2×2 factorial design with capacity and edge-channel width matched: **GAT** ($\neg T \neg Q$, 437,496 parameters), **HGT** ($T \neg Q$, 434,620), **GAT-QoS** ($\neg T Q$, 429,992), **HGT-QoS** ($T Q$, 434,620). One CPU sweep, twelve LOSO folds, five seeds, Application population. Holm correction across the three orthogonal quantities. Cell means: 0.563, 0.548, 0.635, 0.622.

Quantity	Contrast	$\Delta\rho$	95% CI	Won	W	p	p_{Holm}
<i>Three orthogonal quantities, Holm-corrected across these three</i>							
Typing (main effect)	averaged over Q	-0.014	[-0.052, +0.023]	4/12	29.0	0.470	0.940
“QoS” inputs (main effect)	averaged over T	+0.073	[+0.013, +0.120]	10/12	13.0	0.043	0.127
Typing $\times$ “QoS” interaction	difference of differences	+0.001	[-0.050, +0.042]	6/12	34.0	0.733	0.940
<i>Simple effects — descriptive, not separately corrected</i>							
Typing, QoS absent	HGT vs. GAT	-0.015	[-0.064, +0.033]	5/12	31.0	0.569	—
Typing, QoS present	HGT-QoS vs. GAT-QoS	-0.013	[-0.054, +0.026]	4/12	27.0	0.380	—
“QoS” inputs, typing absent	GAT-QoS vs. GAT	+0.072	[+0.028, +0.109]	10/12	9.0	0.016	—
“QoS” inputs, typing present	HGT-QoS vs. HGT	+0.073	[-0.002, +0.136]	10/12	19.0	0.129	—

the weighted in-degree. The closed-form controls show the same pattern: unweighted betweenness (0.591) and constant topic weights (0.595) match or exceed QoS-weighted betweenness (0.553), and permuting QoS profiles across topics has no effect ($\Delta = -0.006$ [-0.025, +0.015], $p = 0.733$). On I^* , there is no evidence that QoS contract content affects ranking; on I_{dyn} , the feature-block gain comes from declared publication rates and payload sizes, and QoS-policy content adds nothing (Section 6.1; Supplementary Section S39).

Table 8: Sensitivity and ablation arms: twelve LOSO folds, five seeds, one CPU invocation with every comparator re-run (each reproduces its published per-seed ρ exactly), Application population. $\Delta\rho$ is paired by fold against the comparator named, with a bootstrap 95% CI; Holm within each registered family (F1: degree features; F2: aggregator; F4: the 2×2 with w_{in} held). $-\text{deg}$: **in_degree** and w_{in} zeroed; $-\text{deg}^*$: also PageRank, closeness and eigenvector centrality, whose correlation with **InDeg** is 0.74–0.77 on average. Zero-shot: mean ρ on the five system models. Registered secondary.

Arm	Comparator	LOSO ρ (I^*)	$\Delta\rho$ [95% CI]	Won	p_{Holm}	$I_{\text{dyn}} / I_{\text{comp}}$	Zero-shot
<i>F1: removing the degree features</i>							
GAT-P-QoS- $-\text{deg}$	GAT-P-QoS (0.748)	0.612	-0.136 [-0.205, -0.072]	1/12	0.0049	0.485 / 0.156	0.821
GAT-P-QoS- $-\text{deg}^*$	GAT-P-QoS	0.587	-0.161 [-0.252, -0.080]	1/12	0.0049	0.462 / 0.138	0.654
GAT-QoS- $-\text{deg}$	GAT-QoS (0.635)	0.365	-0.270 [-0.344, -0.199]	0/12	0.0015	0.295 / -0.047	0.810
<i>F2: sum aggregation (Graph Isomorphism Network with Edge features — GINE, 434,123 parameters) instead of softmax attention</i>							
GIN-P-QoS	GAT-P-QoS	0.732	-0.016 [-0.055, +0.017]	7/12	0.910	0.606 / 0.407	0.806
GIN-P-QoS- $-\text{deg}$	GAT-P-QoS- $-\text{deg}$	0.721	+0.108 [+0.027, +0.197]	8/12	0.157	0.602 / 0.428	0.810
GIN-P-QoS- $-\text{deg}^*$	GAT-P-QoS- $-\text{deg}^*$	0.711	+0.124 [+0.031, +0.230]	9/12	0.157	0.589 / 0.438	0.776
<i>F4: matched 2×2 with w_{in} held (cells GAT+w_{in} 0.628, HGT+w_{in} 0.569, GAT-QoS 0.635, HGT-QoS 0.622)</i>							
Typing (main effect)	averaged over Q	—	-0.036 [-0.079, +0.004]	2/12	0.330	—	—
“QoS” inputs (main effect)	averaged over T	—	+0.030 [-0.023, +0.085]	9/12	0.330	—	—
Interaction	difference of differences	—	+0.046 [-0.009, +0.101]	9/12	0.330	—	—

What the dependency-graph learner needs. Each learner receives in-degree and w_{in} as node properties, and on the dependency graph, it also aggregates the dependent neighborhood. This means its similarity to **InDeg** could come from either source. Ablation experiments (Table 8) separate these effects. Removing the two degree columns reduces **GAT-P-QoS** by 0.136, and removing the three centralities most correlated with in-degree reduces it by 0.161 (F1). This shows that the approach to the reference depends on the degree features. Message-passing theory predicts this: softmax attention normalizes over neighbors and computes a weighted mean, so it cannot count them, whereas a sum aggregator can [69, 70]. A GINE network with the same parameter budget supports this, keeping 0.721 and 0.711 without the same columns, only 0.01–0.02 below its full-feature score. However, the aggregator contrasts are not significant after Holm correction (+0.108 and +0.124, $p_{\text{Holm}} = 0.157$), so the study does not claim an aggregator effect. No dependency-graph learner, with or without degree features, is equivalent to **InDeg** within ± 0.05 (equivalence bounds 0.063–0.276). In zero-shot, the degree columns matter little (0.810–0.821 without them), because the system models most of the ranking by separating inert from active components (Section 6.3).

Direction versus dependency semantics. On the raw multigraph a forward GAT never delivers a message to an Application (Section 4.1), so the +0.113 gain of **GAT-P-QoS** over **GAT-QoS** could come from edge direction rather than from dependency semantics. A control passes every raw-graph edge in both directions, with shared weights and an unchanged parameter count (**GAT-QoS-R**; Amendment 16, Table 9). It reaches 0.676: direction alone recovers +0.041 of the gain, which is not significant (Holm $p = 0.064$),

and GAT-P-QoS still exceeds it by +0.072 [+0.034, +0.113] on 10 of 12 folds (Holm $p = 0.014$). The derived dependency graph therefore adds beyond edge direction. Reverse edges also cost transfer: GAT-QoS-R reaches 0.744 zero-shot, against 0.805 for GAT-QoS and 0.806 for GAT-P-QoS (Table 10).

Table 9: Amendment 16 arms: twelve LOSO folds, five seeds, one CPU invocation with every comparator re-run, Application population. $\Delta\rho$ is paired by fold against the comparator named, with a bootstrap 95% CI; Holm within each registered family (F8: direction; F9: hybrids with the corrected prior; F10: hybrids with the InDeg prior). GAT-QoS-R: GAT-QoS with every raw-graph edge also passed in reverse (shared weights, 429,992 parameters). -AP: the Topo-QoS prior with its articulation term restored; Topo-QoS-AP is that corrected baseline. The re-run learned comparators reproduce their published per-seed ρ exactly, except Hybrid-HGT, which differs by at most 1.2×10^{-4} per seed and is not a comparator here. $I_{\text{dyn}} / I_{\text{comp}}$: per-seed means from saved predictions. Zero-shot: mean ρ on the five system models (— where not run). Registered secondary.

Arm	Comparator	LOSO ρ (I^*)	$\Delta\rho$ [95% CI]	Won	p_{Holm}	$I_{\text{dyn}} / I_{\text{comp}}$	Zero-shot
<i>F8: edge direction versus dependency semantics</i>							
GAT-QoS-R	GAT-QoS (0.635)	0.676	+0.041 [+0.000, +0.087]	10/12	0.064	0.547 / 0.299	0.744
GAT-P-QoS	GAT-QoS-R	0.748	+0.072 [+0.034, +0.113]	10/12	0.014	0.597 / 0.280	—
<i>F9: hybrids with the corrected prior</i>							
Hybrid-GAT-AP	Topo-QoS-AP (0.533)	0.669	+0.136 [+0.076, +0.198]	11/12	0.0059	0.577 / 0.561	0.668
Hybrid-HGT-AP	Topo-QoS-AP	0.640	+0.107 [+0.060, +0.153]	11/12	0.0073	0.552 / 0.553	0.702
Hybrid-GAT-AP	GAT-QoS	0.669	+0.034 [−0.047, +0.118]	7/12	0.940	—	—
Hybrid-HGT-AP	HGT-QoS (0.622)	0.640	+0.018 [−0.067, +0.100]	8/12	0.940	—	—
<i>F10: hybrids with the InDeg prior (within ± 0.012 of InDeg, 0.764)</i>							
GAT-QoS+InDeg	GAT-QoS	0.763	+0.128 [+0.063, +0.204]	11/12	0.0049	0.657 / 0.563	—
HGT-QoS+InDeg	HGT-QoS	0.759	+0.137 [+0.074, +0.211]	10/12	0.0049	0.657 / 0.589	—

Relation typing under matched capacity. HGT-QoS is within 0.013 of GAT-QoS (Table 7). On the raw multigraph, this does not say much about typing, since no message gets an Application there: a gradient-boosted model using the same per-node features but no graph (**GBM-Feat**) achieves 0.642, similar to GAT-QoS, and removing HGT’s reverse pass (the only way messages get Applications) has no effect (HGT-QoS-U, −0.010, $p = 0.91$). On the dependency graph, where messages do reach Applications, HGT-P-QoS gets 0.514 with a mean within-fold seed SD of 0.254, compared to 0.748 and 0.030 for GAT-P-QoS. This is for one fixed configuration (width 100), and averaging predictions from five seeds raises it to 0.618. We interpret this as seed instability for that configuration, not as evidence against relational typing in general. **The registered selection rule.** The plan selected every hyperparameter by nested selection, which was not used in the published results (Section 5.3). When applied later on the stage-1 grid, it changes HGT-QoS from 0.622 to 0.677 (+0.055) and GAT-P-QoS from 0.748 to 0.693 (−0.054), neither of which is significant (Holm $p = 0.259$). Nested HGT-QoS versus Topo-QoS is +0.123 on 9 of 12 folds, still not significant (Holm $p = 0.192$). Using the registered rule does not change the main result. Table 5 shows the fixed, published configuration; The nested-selection run is a registered secondary sensitivity analysis. The search harness also stops early on a held-out training scenario instead of the published node-level split. On the three folds where it chose the published hyperparameters, it does not reproduce the published scores (differences up to 0.10), so these differences are not due to hyperparameter selection alone.

6.3. RQ3: Zero-Shot Transfer to Models of Open-Source Systems

Summary. On five hand-crafted models of open-source systems, the best learned rankers achieve about 0.81 zero-shot ranking for I^* (GAT-P-QoS 0.806, GAT-QoS 0.805), much higher than the training-free baseline (0.526). However, their agreement on the active stratum is weak ($\rho_{>0} \leq 0.342$). The reference rankings are even higher (transitive dependents 0.938, $\rho_{>0} = 0.871$; direct dependents 0.863), since in these models, most of I^* is just reachability. These models are structurally dissimilar; the first author wrote all five, so these results reflect transfer to these models, not to real-world systems. The baseline prior reduces transfer: both hybrids perform worse than their base learners (0.695 and 0.662 compared to 0.760 and 0.805), and correcting the prior’s defect does not change this (0.702 and 0.668; Table 9).

The **Reach** reference achieves $\rho = 0.938$ and $\rho_{>0} = 0.871$ on the five models: 0.836–0.997 on the three original publish–subscribe systems and 0.966–0.998 on the two RPC systems converted to pub-sub meshes. These values are higher than anything **Reach** achieves on the synthetic folds (0.732, $\rho_{>0} = 0.286$), and this difference is due to structural factors. Half of the system models’ Applications have zero simulated impact

Table 10: Zero-shot transfer to hand-authored models inspired by five open-source systems (Application population). Models are evaluated out-of-distribution without fine-tuning. $\rho_{>0}$ denotes rank correlation restricted to active components ($I^* > 0$). PR-AUC measures critical-set identification quality. All rows are scored on the same labels (the learned rankers’ zero-shot label files). The training-free rows are not trained on anything, so “zero-shot” applies only to the learned rows. Reference rows restate I^* ’s propagation rule and are not predictors. Underlined: best predictor per column. Intervals are percentile bootstraps over only five systems (126 distinct resamples); they are descriptive, not inferential, and no test is applied to the zero-shot contrasts, whose training-free comparator carries the articulation defect of Section 5.2.

Ranker	Evaluation Substrate	Mean ρ [95% CI]	Active $\rho_{>0}$	PR-AUC
<i>Reference: restatements of I^*’s propagation rule (not predictors)</i>				
Reach	Dependency graph	0.938 [0.879, 0.991]	0.871	0.933
InDeg	Dependency graph	0.863 [0.734, 0.952]	0.321	0.752
<i>Training-free baseline</i>				
Topo-QoS	Dependency graph	0.526 [0.357, 0.699]	−0.088	0.474
<i>Learned and hybrid, raw multigraph</i>				
HGT-QoS	Raw multigraph	0.760 [0.714, 0.819]	0.236	0.713
GAT-QoS	Raw multigraph	0.805 [0.759, 0.868]	0.319	0.790
Hybrid-HGT	Raw multigraph	0.695 [0.643, 0.730]	0.210	0.602
Hybrid-GAT	Raw multigraph	0.662 [0.597, 0.727]	0.185	0.600
<i>Learned, dependency graph</i>				
GAT-P-QoS	Dependency graph	<u>0.806</u> [0.785, 0.829]	<u>0.342</u>	<u>0.838</u>

(0.51 on average, compared to 0.31 on the folds), and their fan-in is more concentrated (Gini 0.65 vs. 0.50). On these graphs, simply separating the inert half from the rest already gives a high full-population ρ , and **Reach**, which is zero for components with no dependents, does this perfectly. The active-stratum $\rho_{>0}$ is more informative here. On this measure, all predictors are weak (learned rankers 0.185–0.342, training-free baseline negative), as is the direct-dependent reference (0.321 [−0.044, 0.687]), while **Reach** (0.871) is much higher. In these models, ordering active components by transitive reach closely matches what I^* computes, showing strong circularity rather than transferable skill. Per system, **GAT-P-QoS** reaches 0.774 on Train-Ticket, 0.790 on Online Boutique, 0.794 on Autoware, 0.832 on Home Assistant, and 0.841 on EdgeX Foundry. This narrow spread comes largely from separating inert from active components; on the active stratum, its mean agreement is $\rho_{>0} = 0.342$. All five models were created by the first author; no second modeler re-derived any of them and no inter-modeler agreement is reported, so the modeler’s discretion in routing topics and synthesizing broker channels is an unquantified threat (Section 7.5). Furthermore, two systems are stylized pub-sub versions of synchronous RPC architectures (Section 5.1). Consequently, these results show that learned rankers can transfer broad topological partitioning across hand-written model styles, but do not provide reliable within-cascade discrimination on active components in new architectures ($\rho_{>0} \leq 0.342$).

6.4. RQ4: Cost

Summary. Measured like for like, on the same graphs in one session (Table 11), one labeling pass of the reachability oracle over the Applications takes 0.01–0.72 s per corpus architecture, the counting path that restates its first wave 17–176× cheaper (median 45×), and the feature extraction every learned ranker needs 4.5–72.5× more (median 16.9×). On generated graphs of up to 5,000 components, the ordering holds, with features at 5.7–18.7× one pass. The queue-flow oracle is the expensive one: 12.7 CPU-hours to label the corpus. Estimated from the processor’s base power (28 W TDP), replacing the dynamic simulation with the rate-weighted reference (Eq. (7)), computed in at most about a millisecond per architecture, avoids an estimated 355.6 Wh for the corpus. For reachability, static counting remains the greenest choice. Cost should be read together with ranking quality: for both I^* and I_{dyn} , no learned alternative exceeds the analytical approximations, which avoid training and run in milliseconds.

Table 11 times each stage on the same graphs in one session, so the ratios compare like with like. The

Table 11: Like-for-like cost, every region timed on the same graph in one session (CPU, one thread, median of 3 runs, once at 4,995 components, where the three-type sweep was not run). Count: Application–Library projection plus InDeg. One I^* pass: seed 42, Applications, the population every ranker is scored on. Sweep: the published labeling run, five seeds over Application, Broker, and Library nodes. Features: the app-layer analysis that produces the learned rankers’ node features. Gate: the system-layer analysis, quality scoring, and 18 anti-pattern detectors (the full detection gate), listed for reference. Corpus row: minimum–median–maximum over the twelve folds.

$ V $	Count (ms)	One I^* pass (s)	Sweep (s)	Features (s)	Gate (s)	Pass : count	Features : pass
249	2.5	0.26	1.63	1.46	2.05	105×	5.7×
499	6.6	0.79	6.59	6.49	10.01	119×	8.2×
999	13.1	3.90	32.42	30.57	50.92	297×	7.8×
1,998	35.5	18.45	157.86	160.73	275.80	521×	8.7×
4,995	226.2	94.58	—	1,768.81	3,072.60	418×	18.7×
Corpus	0.4–15.6	0.01–0.72	0.09–4.98	0.07–52.25	0.16–81.48	17–45–176×	4.5–16.9–72.5×

full detection gate (system-layer analysis plus 18 anti-pattern detectors) and the five-seed labeling sweep over three node types are listed separately, because no ranker requires them at ranking time. Timed this way, each stage keeps its place in the ordering. The counting path is the cheapest, one I^* pass over the scored population is next, and the app-layer feature extraction that the learned rankers read is the most expensive, at 4.5–72.5× one pass on the corpus. The articulation and Connectivity Degradation Index phase accounts for 88–91% of it (Enterprise, and the 2,000-component graph), and it is largest on Enterprise, whose Rule 1 projection is nearly complete. This cost belongs to the chosen feature set, not to learned ranking as such; a learner restricted to cheaper features would avoid that phase, at an accuracy this study did not measure. Where the I^* ranking itself is wanted, running the oracle is therefore cheaper than approximating it with a learned ranker at every size measured, and the count is cheaper still, by one to two orders of magnitude, because it restates the oracle’s first wave; its practical advantage is cost, not correctness over the oracle it approximates. In the replication repository, timing profiles show that inference itself is negligible (tens of milliseconds at 2,000 components), and that the count scales to 10,000 components (1.1 s) and **Reach** less well (23 s), because it computes a transitive closure; complete timing artifacts and execution scripts are in `docs/research/jss/experiments/rq4-cost.md`. The expensive oracle is I_{dyn} : labeling the corpus took 12.7 CPU-hours (about 7 s per fault run and seed), from 0.01 CPU-hours for Healthcare to 5.88 for Financial Trading (median 0.41). Fault runs are independent across components and seeds, so labeling parallelizes, but a seed-parallel run is still bounded by its slowest seed (about 76 min for Financial Trading). The rate-weighted reference (Eq. (7)) approximates it in at most about a millisecond per architecture and the learned approximation of Section 6.1 in seconds. Hardware power counters (e.g., Running Average Power Limit — RAPL, or external power meters) were not instrumented, so we estimate energy as wall-clock time multiplied by the processor’s base power (28 W Thermal Design Power — TDP for the 13th Gen Intel Core i7-1370P; `reproduce/energy_estimate.py`). Under this estimate, labeling the corpus with I_{dyn} took an estimated 355.6 Wh (≈ 1.28 MJ), the five-seed I^* labeling sweep 11.1 s (0.086 Wh, 310 J), and the full detection gate 106.9 s (0.83 Wh, 2.99 kJ). Summed over the twelve folds, the app-layer feature extraction that the learned rankers read took 70.0 s (about 0.54 Wh), and the dependency count 0.037 s (about 1 J; 0.4–15.6 ms, or 0.01–0.44 J, per architecture). From a Green Software Engineering and Green CI/CD perspective [21, 22, 23], running full dynamic simulations, or extracting the features that learned rankers need, on every commit is the high-overhead, energy-intensive path; neural inference itself is negligible. A learned approximation could repay its computational cost, along with its carbon footprint, only by displacing repeated executions of an expensive simulator like I_{dyn} , and here the rate-weighted reference (Eq. (7)) already does so without training; for cheap reachability analysis, simple static graph queries remain the greener choice.

7. Discussion, Threats to Validity, and Limitations

7.1. Dependency Representation versus Model Complexity

The central finding of this study is that, for the reachability and queue-flow simulators, explicit dependency representations contributed more to ranking performance than model complexity. Deriving a dependency

graph from publish–subscribe deployment manifests enabled both analytical and learned approaches: attention-based learners improved on it compared with the raw multigraph (GAT-P-QoS 0.748 vs. GAT-QoS 0.635), and rankings computed directly from it, such as the direct-dependent count (0.764; not statistically distinguishable from GAT-P-QoS, 0.748) and the rate-weighted first-order approximation (0.830), were not distinguishable from, or exceeded, the best learned models (0.748 and 0.799). Pure learners on the raw multigraph did not outperform the training-free baseline, because messages never reached the scored components. Explicit dependency derivation is therefore not merely a preprocessing step for graph learning; in the evaluated corpus, it is the principal source of predictive signal, and this holds against a control for edge direction: a raw-graph GAT that passes every edge in both directions recovers +0.041 of the +0.113 (not significant), and the derived graph adds +0.072 beyond it (Holm $p = 0.014$; Section 6.2). The multi-criteria simulator is the exception: there, raw degree and betweenness, whose terms align with fragmentation, ranked highest.

7.2. When Learning Adds Value, and Relative to What

The study does not show that learning is never useful; it shows where learning adds value and against which comparator. Relative to the training-free baseline, learning helped in two ways. First, hybrids that correct a closed-form prior significantly outperformed that baseline (+0.103 and +0.130, 11 of 12 folds), and in an exploratory comparison they were the most consistent learned models across the three simulators. This gain is modest evidence for learning: the baseline is the weakest analytical ranker, carries the articulation defect, and is nested in the hybrids as their prior, and neither hybrid differs from its own base learner (+0.035, $p = 0.73$; +0.048, $p = 0.30$). Correcting the prior’s defect does not change this: the corrected-prior hybrids beat the corrected baseline by +0.136 and +0.107 but their base learners by only +0.034 and +0.018 (Holm $p = 0.94$), so the gain belongs to the comparator, not the correction. With the direct-dependent count as the prior, both learners reproduce that count to within ± 0.012 . Second, learned models transferred zero-shot to unfamiliar architectures better than the baseline (≈ 0.81 vs. 0.53). Relative to analytical references built on the same dependencies, however, no learned model showed additional skill. The references were indistinguishable from or exceeded the learned models in-distribution and ranked higher in transfer (0.863–0.938), and ordering among components that actually propagate failures remained weak for every learned model ($\rho_{>0} \leq 0.342$). Learning is therefore likely to contribute most where impact depends on mechanisms that no available analytical approximation captures, a condition this corpus did not exercise. A per-regime breakdown of these results by architecture type is given in Supplementary Section S34.

How message passing contributes. On the raw multigraph, learning mainly used node properties. Three checks explain how these learned rankers worked. First, gradient-boosted decision trees using only per-node features (no graph message passing, **GBM-Feat**) reach $\rho = 0.642$ under LOSO, similar to GAT-QoS (0.635). Second, in the raw multigraph, all structural links point away from Applications, so standard forward GAT convolutions do not send messages to the scored nodes—removing all edges does not change their outputs. Third, HGT reaches Applications only through reverse edge passes, and removing them has no effect (HGT-QoS-U, -0.010 , $p = 0.91$). So, on the raw multigraph, neural models acted as non-linear combiners of precomputed centralities and degree metrics, not as relational message-passing models. On the derived dependency graph, directed edges go from dependents to dependencies: three layers of message passing gather information from the true dependent neighborhood, letting GAT-P-QoS outperform GBM-Feat by +0.106. Since every learner also gets an in-degree feature (Section 3.5), this margin does not mean attention finds the dependent count on its own, and ablation tests show it does not: without the degree columns, GAT-P-QoS loses 0.136, while a sum-aggregation network of the same size loses 0.011 (Section 6.2).

Homogeneous versus heterogeneous architectures. On the raw multigraph, where message passing into Applications was inactive, adding typing made no measurable difference (main effect -0.014 , interaction +0.001). On the dependency graph, where message passing is active, the heterogeneous transformer (HGT-P-QoS) was unstable across seeds in the tested configuration (width 100, fixed setup; mean within-fold seed standard deviation 0.254, mean $\rho = 0.514$), while the homogeneous GAT-P-QoS was stable (0.030, $\rho = 0.748$). This means that, in this setup, relation-specific parameters did not improve accuracy, though they reduced stability. This does not mean typing is always harmful. Of the dependency-graph and raw-graph learners, only HGT-QoS and GAT-P-QoS were run under the registered selection rule (Section 6.2), so HGT-P-QoS ran only in its fixed configuration; averaging five seeds’ predictions raises HGT-P-QoS to 0.618

(still below the references), and denser topologies or architectures with messages reaching scored nodes may behave differently.

Why the learned queue-flow approximation does not exceed the formula. The gradient-boosted approximation (GBM-P-QoS→dyn, 0.799) outperforms the unweighted first-order expansion (0.706) because it reads declared publication rates and payload sizes, which I_{dyn} uses and the unweighted expansion ignores. Once rates enter the analytical expansion (Eq. (7)), it reaches 0.830 and exceeds the learned approximation (+0.031, 10 of 12 folds; exploratory). The attention-based approximation (GAT-P-QoS→dyn, 0.598) received neither rates nor payloads, and it scores within 0.008 of the same model trained on I^* . Because the gradient-boosted approximation already received the declared rates and payloads, its deficit is not a matter of missing inputs. Whether a learned model given the rate-weighted expansion itself as an input, or trained to correct it, can exceed the formula is left to future work (Section 7.6).

7.3. Restatement versus Prediction

Several of the highest-scoring rankings largely restate a simulator’s propagation rule: the direct-dependent count and the first-order expansion for the reachability simulator, and the rate-weighted first-order approximation for the queue-flow simulator. The reference criterion (Section 4.4) distinguishes recovering a simulator’s definition from predicting its outcomes, and it changes how both analytical and learned results should be read. A learned model that approaches a reference has learned the simulator’s first-order rule, not necessarily anything beyond it. Because simulation-labeled benchmarks are common in architecture analysis, the criterion offers a general check for evaluations of this kind: before crediting a learned model with predictive skill, compare it against the simplest ranking that restates the simulator’s own computation.

7.4. Practical Consequences

Choosing a ranker by simulator cost. The usefulness of a ranker depends on how costly the simulator it approximates is. If the simulator is inexpensive, like the reachability simulator I^* (0.01–0.72 s per pass over a corpus architecture’s Applications; Section 6.4), practitioners can compute its ranking directly from the manifest. Publish–subscribe afferent coupling, a direct-dependent count used in architecture research for two decades [54, 55], is not statistically distinguishable from any graph neural network tested at a fraction of the cost; because it restates the simulator’s first wave (Section 7.3), its value is cost, not predictive skill. If the simulator is expensive, like the queue-flow simulator I_{dyn} (12.7 CPU-hours to label the corpus), a rate-weighted first-order expansion computed from declared publication rates approximates it at $\rho = 0.830$ in milliseconds and exceeds a gradient-boosted approximation trained on its labels (0.799; exploratory). This study does not show that either simulator, or any of its approximations, predicts real outages.

Choosing a ranker by failure notion. Table 12 summarizes the evidence by oracle. Practitioners must interpret these recommendations as conditional on simulation fidelity. If the discrete-event or cascade simulator does not capture specific physical failure modes (e.g., partial network partitions or disk thrashing), ranking efficacy on physical clusters could diverge. On the reachability oracle I^* , the best learned ranker is the dependency-graph GAT (GAT-P-QoS, 0.748), followed by the hybrids (0.657–0.683); all require training, and none is statistically distinguishable from the direct-dependent reference (0.764). Where the reachability ranking itself is wanted, running I^* directly is the simplest and cheapest option at corpus size. Where the queue-flow notion of impact matters (I_{dyn}), the rate-weighted first-order expansion (0.830) is the recommended ranker; it exceeds the learned approximation trained on queue-flow labels (0.799; exploratory), and this is the only simulator on which declared information carries measurable signal beyond the dependency structure, from publication rates and payload sizes rather than from QoS policies or QoS-weighted scores. Where failure involves fragmentation and throughput loss (I_{comp}), training-free topological scores (Topo-QoS, $\rho = 0.702$) rank highest because their betweenness term aligns with fragmentation (the articulation term is zero; Section 5.2); among learned models, pure neural rankers collapse (≤ 0.334), while hybrids retain robustness ($\rho = 0.585$). On unfamiliar topologies like the five system models, learned rankers transfer zero-shot at about 0.81 against 0.526 for the training-free baseline. Still, active-stratum agreement is weak ($\rho_{>0} \leq 0.342$), and the result rests on five models by one author.

Mitigating Alert Fatigue with a Two-Tiered Triage Protocol. Using a top-20% threshold on the direct-dependent count (InDeg) recovers about half of the true top-20% set on I^* (0.49), while the rest of

the flagged items are not in that set; the best learned ranker does no better (GAT-P-QoS, 0.48). Figure 5 shows the trade-off: recovering 80% of the critical set takes the top 45% by InDeg (0.83) or the top 40% by GAT-P-QoS (0.81), Topo-QoS does not reach this level within the top 50%, and no predictor reaches 90% within the top 50%. To catch most critical components, a gate would need to flag nearly half the system, making these rankers unsuitable as automated pass/fail filters. In a system with over 100 services, flagging 40–50 would overwhelm developers with alerts.

To operationalize these rankings constructively, we propose a conceptual *two-tiered triage protocol* as an architecture inspection heuristic, rather than an automated blocking gate. We emphasize that this protocol is an architectural proposal requiring human-factors and operational workflow validation before industrial adoption:

1. **Tier 1 (Commit-stage triage):** When reviewing pull requests, compute the direct-dependent count (about 11 ms, an estimated 0.086 mWh, 0.31 J) and the rate-weighted first-order expansion (Eq. (7); about a millisecond; an exploratory result, not confirmed on new data). Show the top 5 components by each as an informational note in the pull request review. This highlights potential cascade risks for peer review without blocking developers or causing alert fatigue.
2. **Tier 2 (Staging and sprint verification):** During nightly builds or sprint architecture reviews, run the simulators on the Tier-1 shortlist within a fixed inspection budget, such as the ten highest-ranked services, and reserve redundant broker routing or chaos fault injection for the components the simulation confirms.

Table 12: Which ranker the evidence supports, by the failure notion the oracle encodes (registered secondary and exploratory; Section 6; conditional on simulation fidelity). Where a reference restating the simulator’s rule exists (Section 4.4), it is recommended because it is cheaper and at least as accurate; it is not credited with predictive skill.

Failure notion (oracle)	Ranker	Evidence
Reachability cascade (I^*)	Run I^* directly	0.01–0.72 s per pass; cheaper than any learned ranker’s features; best predictor GAT-P-QoS $\rho = 0.748$, not distinguishable from the direct-dependent count; equivalence not established
Queue-flow delivery loss (I_{dyn})	Rate-weighted expansion (training-free; exploratory)	$\rho = 0.830$ in milliseconds (Eq. (7); exploratory); gradient-boosted approximation 0.799; unweighted expansion 0.706; GNN approximation 0.598
Fragmentation and throughput (I_{comp})	Topo-QoS (training-free)	$\rho = 0.702$, raw total degree 0.719 (its betweenness term aligns with fragmentation; the articulation term is zero); among learned models, only hybrids retain signal ($\rho = 0.585$), while pure learners collapse (≤ 0.334)
Unfamiliar topology	GAT-P-QoS or GAT-QoS	$\rho = 0.806 / 0.805$ vs. baseline 0.526; dependency counts rank higher (0.863–0.938); $\rho_{>0} \leq 0.342$; five single-author models

Cost and Green CI/CD. As shown in Table 11, the count is the cheapest ranker, followed by a single pass of I^* . Feature extraction for learned models is the most expensive, costing 4.5 to 72.5 times as much as a single pass on the corpus. Where the reachability ranking is wanted, running I^* or counting dependents is simpler and cheaper than any learned approximation. The same holds for the queue-flow simulator, where the rate-weighted expansion replaces an estimated 355.6 Wh (12.7 CPU-hours at 28 W TDP) of simulation for the corpus with a computation of milliseconds. From a Green CI/CD perspective [23, 22, 21], training-free dependency analysis is therefore the lower-energy choice at both commit and staging time. The energy figures are estimates derived from processor TDP, not measurements (Section 7.5).

7.5. Threats to Validity

Construct validity and oracle circularity. All labels are simulated, and no ranker is validated against observed failures in physical deployments. We conducted no empirical fault injection (e.g., chaotic network

partitions, process crashes, or disk exhaustion) on running distributed clusters. Because all rankers and references are evaluated against simulator-generated labels, the findings establish what can be predicted within the evaluated simulation framework, not what necessarily predicts real-world failures; external validation remains necessary. Discrete-event simulations omit complex real-world distributed runtime dynamics: kernel TCP buffer bloat, socket starvation, thread pool exhaustion, garbage collection freezes, asymmetric network partition drops, and cascading retry storms. In physical deployments, such dynamics can decouple the impact of failure from the dependency structure: a low-degree service that blocks on a slow disk, for example, can exhaust upstream connection pools. In such cases, structural rankings, whether analytical or learned, may misestimate criticality, which is why evaluating SaG under live fault injection is the study’s first priority for future work (Section 7.6). The circularity between the primary oracle and the rankers is analyzed in Section 4.4: the dependency counts are references for I^* , the rate-weighted expansion is a reference for I_{dyn} , closed-form centrality is term-aligned with I_{comp} , and the learned rankers inherit part of the overlap through their degree features, which ablation experiments show they depend on. Inert components are separated by construction, which is why $\rho_{>0}$ is reported beside ρ . We decided to reclassify the counts as references after all results were finalized, and this change did not affect any numbers. The two other oracles bound, but do not remove, the threat: I_{dyn} shares the topology and the first-order effect, and I_{comp} ’s weights encode a declared rather than an elicited priority vector. No learned ranker exceeded the analytical references on any simulator: the learned queue-flow approximation outperformed only the unweighted expansion, and the rate-weighted expansion, itself a reference for I_{dyn} , exceeded it. More fundamentally, all three oracles are well approximated by low-order functions of the dependency graph: a first-order expansion recovers 0.808 of I^* and a rate-weighted one 0.830 of I_{dyn} , and I_{comp} ’s terms track degree and betweenness. The benchmark therefore cannot exhibit a regime in which learning exceeds an analytical ranking aligned with the simulator, and the absence of such a regime here is a property of these oracles, not evidence that none exists. Representation and direction were confounded for the learned rankers, because forward message passing on the raw multigraph never reaches Applications. A reverse-edge control (Amendment 16) separates them: direction accounts for +0.041 of the +0.113 gain (not significant), and the derived graph adds +0.072 beyond it (Holm $p = 0.014$). The control shares one set of convolution weights across both directions; a direction-typed variant was not tested. The evaluation is confined to Applications, so dependency Rules 2–4 and 6, which govern brokers and hosts, are untested.

Internal validity, reproducibility, and drift. No ranker reads simulator output, which regression tests verify; the raw-graph learners do read the structural edges the oracles traverse, which is an overlap of inputs, not of labels. We held model capacity, depth, and early stopping constant across comparison arms. The registered selection rule was not applied to the published learned results, a deviation disclosed in Section 5.3; a post hoc sensitivity check applies it to HGT-QoS and GAT-P-QoS (Section 6.2), and every other statement regarding a learned ranker is conditional on one fixed configuration. The learned rankers are also seed-sensitive. Across the five seeds of a fold, the mean within-fold standard deviation of ρ is 0.099 for HGT-QoS, 0.254 for HGT-P-QoS, 0.024 for GAT-QoS and 0.030 for GAT-P-QoS, and single HGT seeds differ from others in the same fold by more than 1.0. These are runs that converge to a ranking anti-correlated with the oracle: one of 60 HGT-QoS seed runs (Telecom RAN) and four of 60 HGT-P-QoS runs (ATM, Industrial SCADA, IoT) have $\rho < 0$, against none for either GAT. They are kept in the per-seed means, as the plan registers, so the HGT rows reflect training instability as much as typing. Averaging the five seeds’ *predictions* instead of their ρ values raises every learned ranker, most of all the unstable ones (HGT-P-QoS from 0.514 to 0.618); the tables report the registered statistic, the mean of per-seed ρ . Across revision cycles and compute devices, learned cells drifted by up to 0.172 (± 0.041 on HGT-QoS), more than several effects this paper reports (the primary contrast, +0.069; the QoS main effect, +0.073; the typing effects, -0.014 and -0.036). Differences between learned rankers smaller than this drift should not be interpreted, whatever their nominal p -value. For that reason, every learned cell in Tables 5, 7 and 10 is reported from one named CPU sweep whose per-seed logs reproduce the published mean exactly, the sensitivity verification sweep re-ran every learned comparator to an exact match, and a hierarchical bootstrap that resamples seeds within folds widens the learned rows’ intervals only slightly. The Connectivity Degradation Index samples nodes with hash-dependent tie-breaking, so PYTHONHASHSEED=0 is fixed. We reconcile all reported values mechanically against released artifacts.

External validity and the single-modeler threat. Synthetic topologies come from a single generator family, which may induce parametric regularities not representative of heterogeneous industrial architectures. The five system models are hand-authored approximations (22–41 Applications) of open-source projects, each written by the first author from public documentation. All five RQ3 results are exposed to this threat, and it is not mitigated: no second modeler authored any model independently, and no inter-modeler reliability or agreement metric is reported. The system models also differ structurally from the synthetic folds: a larger share of their Applications has zero simulated impact, and their fan-in is more concentrated, favoring **Reach** and inflating full-population ρ . Furthermore, two of the five (Train-Ticket and Online Boutique) are synchronous RPC systems re-expressed as asynchronous publish–subscribe meshes by decomposing each RPC call into reciprocal request and reply topics mediated by broker intermediaries. While this translation faithfully mirrors bidirectional dataflow, mapping synchronous request–reply call graphs into decoupled pub-sub event topics may introduce semantic distortions into dependency relations, because it neglects synchronous blocking, client-side timeouts, and upstream thread starvation. We did not evaluate architectures with tens of thousands of components; we timed the counting path up to 10,000 components (Section 6.4), and we did not evaluate the learned rankers or the oracles. We did not reproduce published learned criticality models (FINDER, DrBC) because they do not support typed multigraphs or pub-sub semantics.

Sustainability and energy estimation validity. Energy figures (Section 6.4) are wall-clock time multiplied by the processor’s 28 W base power (`reproduce/energy_estimate.py`), not readings from RAPL counters or power meters. They ignore frequency scaling, turbo power above base (up to 64 W on this processor), DRAM and idle power, and attribute whole-package power to single-threaded work. They support order-of-magnitude comparisons between milliseconds and hours of computation, not energy or emissions accounting.

Conclusion validity. The only confirmatory results are the plan’s two co-primary contrasts, both null (Section 5.3). Everything else, including the two hybrid contrasts, is registered secondary or exploratory and was designed after the primary null was known. LOSO folds share ten of eleven training scenarios, so the Wilcoxon p -values are nominal and anti-conservative; the Nadeau–Bengio corrected test, which accounts for the overlap, leaves the hybrid contrasts significant and the primary null. For training-free rankers, the folds are simply twelve scenarios from one generator, so the effective sample is small, and with twelve folds a bootstrap CI over folds is coarse; over five system models it is indicative only. Statistical equivalence at ± 0.05 is not established for any pair; where the paper says one ranker “matches” another, it means the two are not significantly different (Section 5.3). With twelve folds the study is underpowered for equivalence at that margin (the 90% interval of the per-fold difference between **GAT-P-QoS** and **InDeg** is ± 0.076). Results are also sensitive to how folds are weighted: weighting by $|V_{\text{app}}|$ raises **Topo-QoS** to 0.596, level with the raw-graph learned rankers (0.595–0.606).

7.6. Limitations and Future Work

Five directions follow from the study’s central question, in order of priority:

1. **Validation against real failures:** controlled fault injection on ROS 2, Kafka and MQTT deployments (for example with Chaos Mesh or ChaosBlade), reference systems with known faults such as Train-Ticket [33], and incident data from industrial publish–subscribe systems. This should be coupled with runtime distributed tracing standards (e.g., OpenTelemetry, W3C Trace Context) to empirically validate whether static declared dependency paths faithfully reflect dynamic runtime message flow trajectories under normal and degraded conditions.
2. **Infrastructure and broker validation:** extending cascade-impact evaluation beyond Applications to message brokers, topics, and compute hosts (evaluating Rules 2–4 and 6), which were outside the Application-scoped evaluation of this study.
3. **Independent modelers:** replication of the five system models by additional modelers, with inter-modeler agreement reported.
4. **Alternative impact models:** testing whether the conclusions hold for simulators with different failure mechanisms, including synchronous call semantics, which the current dependency rules do not model, and oracles with non-first-order mechanisms such as backpressure onto publishers, retry amplification and heterogeneous service capacity, where an aligned analytical approximation may not exist.

1134 **5. When learning exceeds analytical methods:** identifying impact-generation mechanisms whose
1135 predictive signal is not captured by explicit dependency structure. This includes learned corrections on
1136 top of the rate-weighted approximation, learned models that receive the rate-weighted expansion itself
1137 as an input, graph neural networks given the same rate and payload information as the gradient-boosted
1138 approximation, and a confirmation corpus generated after the analysis is frozen.

1139 Further extensions are automatic extraction of architecture graphs from ROS 2 launch files, Kubernetes man-
1140 ifests and Helm charts; integration with service mesh telemetry and distributed tracing (e.g., OpenTelemetry,
1141 Jaeger) for runtime dependency drift detection; hardware-measured energy accounting (for example via
1142 RAPL); a queue-flow run with QoS policies disabled; a tie-aware listwise loss with scenario-level validation; a
1143 full hyperparameter search for every learned model; and validation of the proposed explanation layer.

1144 8. Conclusion

1145 This paper presented Software-as-a-Graph (SaG), a pre-deployment framework that derives explicit
1146 dependency graphs from publish-subscribe deployment manifests and supports analytical, hybrid, and
1147 learned approaches to cascade-impact ranking. Using twelve synthetic architectures, three failure simulators
1148 and five stylized models of open-source systems, the study examined when graph learning improves on
1149 rankings derived directly from explicit architectural dependencies. The primary registered contrast was null,
1150 and the findings below are registered secondary or exploratory.

1151 For the reachability and queue-flow simulators, explicit dependency representations provided most of
1152 the predictive signal. Attention-based learners benefited from the derived dependency graph ($\rho = 0.748$
1153 vs. 0.635 on the raw multigraph), but simple analytical rankings built on the same dependencies were not
1154 distinguishable from, or exceeded, the learned models. Counting direct dependents ($\rho = 0.764$) was not
1155 statistically distinguishable from the best learned model on the reachability simulator (0.748; equivalence at
1156 ± 0.05 not established). On the expensive queue-flow simulator, a rate-weighted first-order approximation
1157 computed in milliseconds (0.830) exceeded a learned approximation trained on 12.7 CPU-hours of simulation
1158 labels (0.799; exploratory). In this corpus, representation choice mattered more than model complexity, and
1159 the learned gain from the derived graph survives a control that passes raw-graph edges in both directions
1160 ($+0.072$, Holm $p = 0.014$).

1161 Learning added value only relative to a weak comparator. Hybrids that correct a closed-form prior
1162 significantly outperformed the training-free baseline, which they nest and which is the weakest analytical
1163 ranker, also after its defect was corrected, but not their own base learners; and learned models transferred
1164 zero-shot to unfamiliar architectures better than that baseline (≈ 0.81 vs. 0.53). In both cases, however,
1165 dependency-based references remained competitive or stronger, and ordering among components that actually
1166 propagate failures remained weak.

1167 These results have implications for both research and practice. For researchers, they show the importance
1168 of evaluating graph-learning approaches against analytical alternatives aligned with the simulator, and
1169 of distinguishing prediction from restatement of a simulator’s rule. For practitioners, they suggest that
1170 training-free dependency analysis provides effective and inexpensive support for prioritizing architectural
1171 review. Learned models can be expected to add value only where no analytical approximation aligned with
1172 the simulator is available. All three simulators studied here admit such an approximation, so this corpus
1173 could not exercise that condition, and no regime was found in which learning exceeded it.

1174 These findings concern simulated failures on synthetic architectures and single-modeler system models,
1175 and their validation against real outages is the natural next step (Section 7.6). More broadly, the study
1176 suggests that the central challenge in architecture-level cascade-impact analysis is not selecting increasingly
1177 sophisticated learning algorithms, but deriving meaningful dependency representations and understanding
1178 when learning adds value beyond them. By making this comparison explicit and by releasing a reproducible
1179 benchmark with reconciled results, Software-as-a-Graph provides a foundation for more principled evaluation
1180 of analytical and learned methods for pre-deployment architecture analysis.

Declarations

CRedit Authorship Contribution Statement. **Ibrahim Onuralp Yigit:** Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Visualization. **Feza Buzluca:** Conceptualization, Methodology, Validation, Writing – review and editing, Supervision.

Declaration of Competing Interest. The authors declare no competing financial interests or personal relationships that could have influenced this work.

Funding. This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Data Availability. The complete replication package—including datasets, simulation harnesses, model checkpoints, analysis scripts, and reproduction workflows—is archived on Zenodo under DOI 10.5281/zenodo.23045204 [87]. The open-source codebase, environment specifications, and reproduction documentation are publicly available on GitHub at <https://github.com/onuralpyigit/software-as-a-graph/tree/jss-submission/docs/research/jss/experiments>. The script `reproduce/reconcile_manuscript.py` mechanically verifies 1,711 reported figures against the released artifacts.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process. During the preparation of this work, the authors used Anthropic’s Claude Opus 5 and Opus 5.5, and Google’s Gemini Flash 3.8, to suggest revisions to the manuscript text, which the authors evaluated, edited, and verified against the results; for LaTeX typesetting and formatting assistance; and for developing analysis scripts in the replication package. They used Grammarly for grammar checking and language copy-editing. The authors reviewed and edited the output as needed and take full responsibility for the content of the published article. The study design, the choice of experiments, the interpretation of results, and all scientific claims are the authors’ own.

References

- [1] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, W. Woodall, Robot operating system 2: Design, architecture, and uses in the wild, *Science Robotics* 7 (66) (2022) eabm6074.
- [2] J. Kreps, N. Narkhede, J. Rao, Kafka: A distributed messaging system for log processing, in: *Proc. 6th Int. Workshop on Networking Meets Databases (NetDB)*, 2011.
- [3] Object Management Group, Data Distribution Service (DDS), Tech. Rep. formal/2015-04-10, version 1.4, Object Management Group (2015).
- [4] OASIS, MQTT version 5.0, OASIS Standard, <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html> (accessed 9 September 2026) (2019).
- [5] N. Dragoni, S. Giallorenzo, A. L. Lafuente, M. Mazzara, F. Montesi, R. Mustafin, L. Safina, Microservices: Yesterday, today, and tomorrow, in: *Present and Ulterior Software Engineering*, Springer, 2017, pp. 195–216.
- [6] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, O’Reilly Media, 2015.
- [7] P. T. Eugster, P. A. Felber, R. Guerraoui, A.-M. Kermarrec, The many faces of publish/subscribe, *ACM Computing Surveys* 35 (2) (2003) 114–131.
- [8] A. E. Motter, Y.-C. Lai, Cascade-based attacks on complex networks, *Physical Review E* 66 (2002) 065102(R).
- [9] S. V. Buldyrev, R. Parshani, G. Paul, H. E. Stanley, S. Havlin, Catastrophic cascade of failures in interdependent networks, *Nature* 464 (2010) 1025–1028.
- [10] R. Albert, H. Jeong, A.-L. Barabási, Error and attack tolerance of complex networks, *Nature* 406 (2000) 378–382.
- [11] A. Avizienis, J.-C. Laprie, B. Randell, C. Landwehr, Basic concepts and taxonomy of dependable and secure computing, *IEEE Transactions on Dependable and Secure Computing* 1 (1) (2004) 11–33.
- [12] L. Bass, P. Clements, R. Kazman, *Software Architecture in Practice*, 3rd Edition, Addison-Wesley, 2012.
- [13] D. E. Perry, A. L. Wolf, Foundations for the study of software architecture, *ACM SIGSOFT Software Engineering Notes* 17 (4) (1992) 40–52.
- [14] J. Garcia, D. Popescu, G. Edwards, N. Medvidovic, Identifying architectural bad smells, in: *Proc. 13th European Conf. on Software Maintenance and Reengineering (CSMR)*, 2009, pp. 255–258.
- [15] R. Kazman, M. Klein, M. Barbacci, T. Longstaff, H. Lipson, J. Carriere, The architecture tradeoff analysis method, in: *Proc. 4th IEEE Int. Conf. on Engineering of Complex Computer Systems (ICECCS)*, 1998, pp. 68–78.
- [16] SonarSource, Clean as you code, SonarQube documentation, <https://docs.sonarsource.com/sonarqube-server/latest/core-concepts/clean-as-you-code/introduction/> (accessed 9 September 2026) (2024).
- [17] S. R. Chidamber, C. F. Kemerer, A metrics suite for object oriented design, *IEEE Transactions on Software Engineering* 20 (6) (1994) 476–493.
- [18] A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds, C. Rosenthal, Chaos engineering, *IEEE Software* 33 (3) (2016) 35–41.
- [19] C. S. Meiklejohn, A. Estrada, Y. Song, H. Miller, R. Padhye, Service-level fault injection testing, in: *Proceedings of the ACM Symposium on Cloud Computing (SoCC ’21)*, ACM, 2021, pp. 388–402. doi:10.1145/3472883.3487005.
- [20] G. Yu, P. Chen, H. Chen, Z. Guan, Z. Huang, L. Jing, T. Weng, X. Sun, X. Li, Microrank: End-to-end latency issue localization with extended spectrum analysis in microservice environments, in: *Proceedings of The Web Conference 2021 (WWW ’21)*, ACM, 2021, pp. 3087–3098. doi:10.1145/3442381.3449905.
- [21] C. Calero, M. Piattini (Eds.), *Green in Software Engineering*, Springer, Cham, Switzerland, 2015. doi:10.1007/978-3-319-08581-4.
- [22] R. Verdecchia, J. Sallou, L. Cruz, A systematic review of Green AI, *WIREs Data Mining and Knowledge Discovery* 13 (4) (2023) e1507. doi:10.1002/widm.1507.
- [23] R. Schwartz, J. Dodge, N. A. Smith, O. Etzioni, Green AI, *Communications of the ACM* 63 (12) (2020) 54–63. doi:10.1145/3381831.

- [24] S. M. Yacoub, H. H. Ammar, A methodology for architecture-level reliability risk analysis, *IEEE Transactions on Software Engineering* 28 (6) (2002) 529–547. doi:10.1109/TSE.2002.1010058.
- [25] T. Zimmermann, N. Nagappan, Predicting defects using network analysis on dependency graphs, in: *Proceedings of the 30th International Conference on Software Engineering (ICSE '08)*, ACM, 2008, pp. 531–540. doi:10.1145/1368088.1368161.
- [26] C. Fan, L. Zeng, Y. Sun, Y.-Y. Liu, Finding key players in complex networks through deep reinforcement learning, *Nature Machine Intelligence* 2 (2020) 317–324.
- [27] C. Fan, L. Zeng, Y. Ding, M. Chen, Y. Sun, Z. Liu, Learning to identify high betweenness centrality nodes from scratch: A novel graph neural network approach, in: *Proc. 28th ACM Int. Conf. on Information and Knowledge Management (CIKM)*, 2019, pp. 559–568.
- [28] G. Khodabandeh, A. Ezaz, M. Babaei, N. Ezzati-Jivan, Utilizing graph neural networks for effective link prediction in microservice architectures, in: *Proceedings of the 16th ACM/SPEC International Conference on Performance Engineering (ICPE)*, 2025, pp. 19–30. doi:10.1145/3676151.3719362.
- [29] R. Premraj, K. Herzig, Network versus code metrics to predict defects: A replication study, in: *2011 International Symposium on Empirical Software Engineering and Measurement (ESEM)*, IEEE, 2011, pp. 215–224. doi:10.1109/ESEM.2011.30.
- [30] V. J. Hellendoorn, P. Devanbu, Are deep neural networks the best choice for modeling source code?, in: *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*, 2017, pp. 763–773. doi:10.1145/3106237.3106290.
- [31] F. Errica, M. Podda, D. Bacciu, A. Micheli, A fair comparison of graph neural networks for graph classification, in: *International Conference on Learning Representations (ICLR)*, 2020, <https://openreview.net/forum?id=HygDF6NFPB>.
- [32] I. O. Yigit, F. Buzluca, A graph-based dependency analysis method for identifying critical components in distributed publish–subscribe systems, in: *Proc. IEEE Int. Conf. on Recent Advances in Systems Science and Engineering (RASSE)*, 2025, pp. 1–8. doi:10.1109/RASSE64831.2025.11315354.
- [33] X. Zhou, X. Peng, T. Xie, J. Sun, C. Ji, W. Li, D. Ding, Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study, *IEEE Transactions on Software Engineering* 47 (2) (2021) 243–260. doi:10.1109/TSE.2018.2887384.
- [34] K. Goseva-Popstojanova, K. S. Trivedi, Architecture-based approach to reliability assessment of software systems, *Performance Evaluation* 45 (2–3) (2001) 179–204.
- [35] S. Becker, H. Koziolk, R. Reussner, The Palladio component model for model-driven performance prediction, *Journal of Systems and Software* 82 (1) (2009) 3–22.
- [36] W. Abdelmoez, D. M. Nassar, M. Shereshevsky, N. Gradetsky, R. Gunnalan, H. H. Ammar, B. Yu, A. Mili, Error propagation in software architectures, in: *Proc. 10th IEEE Int. Software Metrics Symp. (METRICS)*, 2004, pp. 384–393.
- [37] P. Popic, D. Desovski, W. Abdelmoez, B. Cukic, Error propagation in the reliability analysis of component based systems, in: *Proc. 16th IEEE Int. Symp. on Software Reliability Engineering (ISSRE)*, 2005, pp. 53–62.
- [38] V. Cortellessa, V. Grassi, A modeling approach to analyze the impact of error propagation on reliability of component-based systems, in: *Proc. 10th Int. Symp. on Component-Based Software Engineering (CBSE)*, Vol. 4608 of LNCS, Springer, 2007, pp. 140–156. doi:10.1007/978-3-540-73551-9_10.
- [39] M. Hiller, A. Jhumka, N. Suri, EPIC: Profiling the propagation and effect of data errors in software, *IEEE Transactions on Computers* 53 (5) (2004) 512–530.
- [40] Y. Papadopoulos, J. A. McDermid, Hierarchically performed hazard origin and propagation studies, in: *Computer Safety, Reliability and Security (SAFECOMP)*, Vol. 1698 of LNCS, Springer, 1999, pp. 139–152. doi:10.1007/3-540-48249-0_13.
- [41] J. Delange, P. Feiler, Architecture fault modeling with the AADL Error-Model Annex, in: *Proc. 40th EUROMICRO Conf. on Software Engineering and Advanced Applications (SEAA)*, 2014, pp. 361–368. doi:10.1109/SEAA.2014.20.
- [42] Y. Gan, Y. Zhang, K. Hu, D. Cheng, Y. He, M. Pancholi, C. Delimitrou, Seer: Leveraging big data to navigate the complexity of performance debugging in cloud microservices, in: *Proc. ACM Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2019.
- [43] L. Wu, J. Tordsson, E. Elmroth, O. Kao, MicroRCA: Root cause localization of performance issues in microservices, in: *Proc. IEEE/IFIP Network Operations and Management Symposium (NOMS)*, 2020.
- [44] C. Lee, T. Yang, Z. Chen, Y. Su, M. R. Lyu, Eadro: An end-to-end troubleshooting framework for microservices on multi-source data, in: *Proc. 45th IEEE/ACM Int. Conf. on Software Engineering (ICSE)*, 2023, pp. 1750–1762. doi:10.1109/ICSE48619.2023.00150.
- [45] S. Zhang, S. Xia, W. Fan, B. Shi, X. Xiong, Z. Zhong, M. Ma, Y. Sun, D. Pei, Failure diagnosis in microservice systems: A comprehensive survey and analysis, *ACM Transactions on Software Engineering and Methodology* (2025). doi:10.1145/3715005.
- [46] S. A. Bohner, R. S. Arnold, *Software Change Impact Analysis*, IEEE Computer Society Press, Los Alamitos, CA, 1996.
- [47] S. Esparrachiar, T. Reilly, A. Rentz, Tracking and controlling microservice dependencies, *ACM Queue* 16 (4) (2018). doi:10.1145/3277539.3277541.
- [48] X. Yang, K. Tang, X. Yao, A learning-to-rank approach to software defect prediction, *IEEE Transactions on Reliability* 64 (1) (2015) 234–246.
- [49] T. J. McCabe, A complexity measure, *IEEE Transactions on Software Engineering SE-2* (4) (1976) 308–320.
- [50] N. Nagappan, T. Ball, Static analysis tools as early indicators of pre-release defect density, in: *Proc. 27th Int. Conf. on Software Engineering (ICSE)*, 2005, pp. 580–586.
- [51] T. Zimmermann, R. Premraj, A. Zeller, Predicting defects for Eclipse, in: *Proc. 3rd Int. Workshop on Predictor Models in Software Engineering (PROMISE)*, 2007.
- [52] V. Bushong, D. Das, A. Al Maruf, T. Cerny, Using static analysis to address microservice architecture reconstruction, in: *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, IEEE, 2021. doi:10.1109/ASE51524.2021.9678749.
- [53] A. Santos, A. Cunha, N. Macedo, Static-time extraction and analysis of the ROS computation graph, in: *2019 Third IEEE International Conference on Robotic Computing (IRC)*, IEEE, 2019. doi:10.1109/IRC.2019.00018.
- [54] R. C. Martin, *Agile Software Development: Principles, Patterns, and Practices*, Prentice Hall, 2003.
- [55] D. Rud, A. Schmietendorf, R. R. Dumke, Product metrics for service-oriented infrastructures, in: *Applied Software Measurement: Proceedings of the International Workshop on Software Metrics and DASMA Software Metrik Kongress (IWSM/MetriKon 2006)*, Shaker Verlag, Aachen, Germany, 2006, pp. 161–174.
- [56] J. Bogner, S. Wagner, A. Zimmermann, Automatically measuring the maintainability of service- and microservice-based systems: A literature review, in: *Proceedings of the 27th International Workshop on Software Measurement and 12th International Conference on Software Process and Product Measurement (IWSM Mensura '17)*, ACM, 2017, pp. 107–115. doi:10.1145/3143434.3143443.

- [57] J. Garcia, D. Popescu, G. Edwards, N. Medvidovic, Toward a catalogue of architectural bad smells, in: Proc. 5th Int. Conf. on the Quality of Software Architectures (QoSA), LNCS 5581, 2009, pp. 146–162.
- [58] D. Taibi, V. Lenarduzzi, On the definition of microservice bad smells, *IEEE Software* 35 (3) (2018) 56–62.
- [59] Z. Li, P. Avgeriou, P. Liang, A systematic mapping study on technical debt and its management, *Journal of Systems and Software* 101 (2015) 193–220.
- [60] J. Humble, D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*, Addison-Wesley, 2010.
- [61] T. L. Saaty, *The Analytic Hierarchy Process: Planning, Priority Setting, Resource Allocation*, McGraw-Hill, 1980.
- [62] L. C. Freeman, A set of measures of centrality based on betweenness, *Sociometry* 40 (1) (1977) 35–41.
- [63] U. Brandes, A faster algorithm for betweenness centrality, *Journal of Mathematical Sociology* 25 (2) (2001) 163–177.
- [64] S. Brin, L. Page, The anatomy of a large-scale hypertextual web search engine, *Computer Networks and ISDN Systems* 30 (1–7) (1998) 107–117.
- [65] M. E. J. Newman, *Networks: An Introduction*, Oxford University Press, 2010.
- [66] A. Varbella, K. Amara, M. El-Assady, B. Gjorgiev, G. Sansavini, PowerGraph: A power grid benchmark dataset for graph neural networks, in: *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, Datasets and Benchmarks Track, 2024, arXiv:2402.02827.
- [67] S. K. Maurya, X. Liu, T. Murata, Graph neural networks for fast node ranking approximation, *ACM Transactions on Knowledge Discovery from Data* 15 (5) (2021) 78:1–78:32. doi:10.1145/3446217.
- [68] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, Y. Bengio, Graph attention networks, in: Proc. Int. Conf. on Learning Representations (ICLR), 2018.
- [69] K. Xu, W. Hu, J. Leskovec, S. Jegelka, How powerful are graph neural networks?, in: *International Conference on Learning Representations (ICLR)*, 2019.
- [70] G. Corso, L. Cavalleri, D. Beaini, P. Liò, P. Veličković, Principal neighbourhood aggregation for graph nets, in: *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 33, 2020, pp. 13260–13271.
- [71] Z. Chen, L. Chen, S. Villar, J. Bruna, Can graph neural networks count substructures?, in: *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 33, 2020, pp. 10383–10395.
- [72] Z. Hu, Y. Dong, K. Wang, Y. Sun, Heterogeneous graph transformer, in: Proc. The Web Conference (WWW), 2020, pp. 2704–2710.
- [73] Q. Lv, M. Ding, Q. Liu, Y. Chen, W. Feng, S. He, C. Zhou, J. Jiang, Y. Dong, J. Tang, Are we really making much progress? Revisiting, benchmarking, and refining heterogeneous graph neural networks, in: Proc. 27th ACM SIGKDD Conf. on Knowledge Discovery and Data Mining (KDD), 2021, pp. 1150–1160. doi:10.1145/3447548.3467350.
- [74] O. Shchur, M. Mumme, A. Bojchevski, S. Günnemann, Pitfalls of graph neural network evaluation, arXiv preprint arXiv:1811.05868 (2018).
- [75] Q. Huang, H. He, A. Singh, S.-N. Lim, A. R. Benson, Combining label propagation and simple models out-performs graph neural networks, in: Proc. Int. Conf. on Learning Representations (ICLR), 2021.
- [76] Z. Ying, D. Bourgeois, J. You, M. Zitnik, J. Leskovec, GNNExplainer: Generating explanations for graph neural networks, in: *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 32, 2019, pp. 9244–9255.
- [77] International Organization for Standardization, ISO/IEC 25010:2023 — systems and software engineering — systems and software quality requirements and evaluation (SQuaRE) — product quality model, Tech. rep., International Organization for Standardization (2023).
- [78] J. Pearl, *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*, Morgan Kaufmann, 1988.
- [79] G. H. Hardy, J. E. Littlewood, G. Pólya, *Inequalities*, 2nd Edition, Cambridge University Press, 1952.
- [80] M. Fey, J. E. Lenssen, Fast graph representation learning with PyTorch geometric, in: *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.
- [81] F. Xia, T.-Y. Liu, J. Wang, W.-S. Zhang, H. Li, Listwise approach to learning to rank: Theory and algorithm, in: Proc. 25th Int. Conf. on Machine Learning (ICML), 2008, pp. 1192–1199.
- [82] Team SimPy, Simpy: Discrete event simulation for Python, Software, <https://simpy.readthedocs.io> (accessed 9 September 2026) (2020).
- [83] B. Efron, R. J. Tibshirani, *An Introduction to the Bootstrap*, Chapman & Hall, 1993.
- [84] F. Wilcoxon, Individual comparisons by ranking methods, *Biometrics Bulletin* 1 (6) (1945) 80–83.
- [85] A. Arcuri, L. Briand, A practical guide for using statistical tests to assess randomized algorithms in software engineering, in: Proc. 33rd Int. Conf. on Software Engineering (ICSE), 2011, pp. 1–10. doi:10.1145/1985793.1985795.
- [86] C. Nadeau, Y. Bengio, Inference for the generalization error, *Machine Learning* 52 (2003) 239–281. doi:10.1023/A:1024068626366.
- [87] İ. O. Yiğit, F. Buzluca, [dataset] software-as-a-graph: Replication package (datasets, generator configurations, simulation harnesses, model checkpoints, and analysis scripts), <https://doi.org/10.5281/zenodo.23045204> (2026). doi:10.5281/zenodo.23045204.